



泉州信息工程学院

Quanzhou University of Information Engineering

本科毕业设计

(2026 届)

题目：基于 Spring Boot+Vue 的个性化音乐
推荐系统的设计与实现

学生姓名	陈诗豪
学 号	2203010338
二级学院	软件学院
专业班级	2022 级软件工程 3 班
指导教师	唐伟彬（校内） 金潇（企业）
填写日期	2026 年 05 月

学位论文独创性声明

本人郑重声明：所呈交的学位论文是本人在指导教师指导下进行的研究工作及取得的研究成果。除文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

学位论文作者签名：

签字日期： 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解泉州信息工程学院有权保留并向国家有关部门或机构送交本论文的复印件和电子版本，允许论文被查阅和借阅。本人授权泉州信息工程学院可以将本学位论文的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

学位论文作者签名：

指导教师签名：

签字日期： 年 月 日

签字日期： 年 月 日

基于 Spring Boot+Vue 的个性化音乐推荐系统的设计与实现

摘 要

数字音乐平台发展很快，海量音乐资源让用户选择空间变大，但信息过载问题也很突出。传统音乐平台主要依靠关键字检索、热门榜单推荐和基于历史行为的协同过滤方法来向用户提供推荐服务，但该类平台在为用户进行推荐时，通常很难同时保证推荐结果具有准确度、实时性和可解释性这三个方面的平衡。本文利用 Spring Boot 和 Vue 技术，开发了一个个性化的音乐推荐系统，用来解决前面提到的问题。系统以 Spring Boot 3.2.5 作为后端中心，使用 MyBatis 进行数据访问，达到高效和灵活地访问数据库。MySQL 可以达到安全和一致地存储结构化业务数据的目的。本文通过使用 Redis 缓存播放列表和播放状态来加快响应速度，并且依靠 MinIO 对音频、封面、头像等对象资源进行管理，从而实现了高效、安全并且可以扩展的非结构化数据存储效果。系统也通过 Spring AI Alibaba DashScope 来完成音频内容分析、特征描述生成和推荐理由生成工作。前端使用 Vue 3、Vite、Pinia、Vue Router 和 Element Plus 来建立用户端和管理端的统一应用，拥有音乐浏览播放、音频上传分析、推荐结果查看、AI 配置管理等功能。经过测试，该系统可以稳定地完成用户端和管理端的主要业务流程，在功能完整性、交互连贯性、模块协同上达到了预期目标。

关键词：个性化音乐推荐；音频特征分析；Spring Boot；AI

Design and Implementation of Personalized Music Recommendation System Based on Spring Boot+Vue

Abstract

Digital music platforms have developed rapidly, offering users an extensive range of musical resources while simultaneously facing significant information overload challenges. Traditional music platforms primarily rely on keyword searches, popular chart recommendations, and collaborative filtering algorithms based on historical user behavior to provide recommendations. However, these platforms often struggle to balance accuracy, real-time performance, and interpretability in their recommendation systems. This paper develops a personalized music recommendation system utilizing Spring Boot and Vue technologies to address these issues. The system employs Spring Boot 3.2.5 as its backend framework, leveraging MyBatis for efficient and flexible database access while ensuring secure and consistent storage of structured business data in MySQL. Response speeds are optimized through Redis caching for playlist and playback status management, complemented by MinIO for handling object resources such as audio files, album covers, and profile images, achieving efficient, secure, and scalable storage solutions. Audio content analysis, feature generation, and recommendation rationale generation are performed using Spring AI's Alibaba DashScope. The frontend utilizes Vue 3, Vite, Pinia, Vue Router, and Element Plus to create unified applications for both user interfaces and administrative panels, featuring music browsing and playback, audio upload analysis, recommendation result viewing, and AI configuration management capabilities. After testing, the system has demonstrated stable execution of core business processes on both the client and management ends, achieving the intended objectives in terms of functional completeness, interactive coherence, and module synergy.

Keywords: Personalized music recommendation; Audio feature analysis; Spring Boot; AI

目录

摘 要	I
Abstract	II
1 前言	1
1.1 研究背景	1
1.2 国内外研究现状	1
1.3 主要研究内容	2
1.4 论文组织结构	3
2 相关技术介绍	4
2.1 Spring Boot 后端开发框架	4
2.2 Vue 3 与 Vite 前端技术	4
2.3 MyBatis 与 MySQL 数据持久化	4
2.4 Redis 缓存与状态管理	4
2.5 MinIO 对象存储	5
2.6 JWT 认证与邮件验证码机制	5
2.7 Spring AI Alibaba DashScope	5
2.8 本章小结	5
3 系统分析	6
3.1 可行性分析	6
3.1.1 技术可行性	6
3.1.2 经济可行性	6
3.1.3 社会可行性	6
3.2 需求分析	6
3.2.1 系统场景描述	6
3.2.2 功能分析	7
3.2.3 角色分析	7
3.3 用例建模	7
3.3.1 系统用例图	7
3.3.2 用例规约	10
3.4 系统主要类模型	14

3.5	本章小结	15
4	系统设计	16
4.1	系统总体结构	16
4.2	系统主要功能模块	17
4.2.1	系统活动图	17
4.2.2	系统时序图设计	22
4.3	系统主要界面架构	26
4.3.1	主要系统类图	26
4.3.2	系统页面关系	27
4.4	数据库设计	29
4.4.1	数据库关系图	29
4.4.2	数据库物理设计	29
4.5	本章小结	36
5	系统实现	37
5.1	系统总体功能说明	37
5.2	功能模块实现	37
5.2.1	用户认证与资料维护模块	37
5.2.2	音频 AI 推荐模块	39
5.2.3	音乐浏览播放与收藏模块	42
5.2.4	社区互动模块	44
5.2.5	内容资源管理模块	46
5.2.6	用户、社区与 AI 管理模块	50
5.3	本章小结	54
6	系统测试	55
6.1	测试方法	55
6.2	功能测试	55
6.3	测试用例设计	55
6.3.1	提交代码判题测试	55
6.3.2	上传音频与分析测试	56
6.3.3	音乐浏览播放与收藏测试	58
6.3.4	社区互动测试	59
6.3.5	内容资源管理测试	60

6.3.6 用户、社区与 AI 管理测试.....	61
6.4 本章小结	63
7 分析、总结与展望	64
7.1 安全与隐私分析	64
7.2 技术经济与成本分析	64
7.3 总结	64
7.4 展望	64
参考文献	66
致 谢	67

1 前言

1.1 研究背景

随着在线的音乐平台的曲库规模的不断的膨胀，对如何从这海量的资源中快速的找出符合自己的独特的个人偏好的作品就已成为当前的数字音乐服务的最核心的、最关键的也是最具挑战性的问题。在对音乐推荐系统的不断深入的研究表明背景下，音频的特征提取、内容的驱动的推荐、机器的学习排序等与混合的推荐机制已逐步成为了音乐推荐系统的重要的发展方向。平台的商业化推进同时，推荐系统的作用也逐渐由“辅助”升级为“核心”，其对的推荐质量不仅直接影响了用户的留存，也更为深刻地关联了内容的传播效率以及整个社区的活跃度^[1]。随着音乐推荐的逐步深入人心，它已不仅仅是一种单纯的算法问题，已逐渐演变为与产品的体验度、社区的生态建设、平台的运营等都紧密地联系在一起的综合性系统工程。

从实用的角度出发，我们就可以发现传统的音乐平台所采取的主要的推荐方式都基本上建立在了对用户的历史点击、播放的次数、用户的收藏记录等行为的数据之上。但在用户的行为都相对充分、平台的数据也基本都能积累起来的基础上，这类方法也就能够在一定的范围内取得较好的效果，但也同样存在着明显的短板。但由于新用户的偏好难以被准确的识别，其所得的推荐结果也容易陷入对热门的歌曲的无限的循环中，难以兼顾到多样性和新颖性的需求等一系列的问题都急需我们去解决。

1.2 国内外研究现状

国外在个性化音乐推荐领域的研究相对来说较早形成体系。部分研究聚焦于音频特征提取技术^[2]，利用 MFCC、STFT 等手段从节奏、旋律和音色中提取关键特征，结合用户播放历史构建动态推荐模型，显著提升了召回率和精确率。另有学者探索了机器学习算法的深度融合^[3]，将监督学习与无监督学习相结合，引入卷积神经网络和循环神经网络处理音乐元数据与用户交互，推动了推荐系统的智能化演进。此外，内容驱动推荐的研究也从基于标签的方法转向深度学习主导^[4]，强调音乐内容的语义理解（如歌词、旋律和情感标签）对提升推荐质量的关键作用。这些研究说明，国外音乐推荐研究已经从早期的纯协同过滤逐渐转向内容与行为融合、深度模型驱动和系统层可解释优化。

国内相关研究则更加注重推荐系统与本土音乐场景、情感分析和系统实现的结合^[5]。诸多学者推出一系列系统或者算法模型。此类系统主要分为两类：一类是基于协同过滤与

标签^[6]的混合推荐系统，聚焦于用户历史行为与歌曲元数据的匹配，解决了传统推荐中的冷启动问题，但功能相对单一，未能深度融合音频内容特征；另一类是基于情感分析^{[7][8][9]}与深度学习^{[10][11][12][13][14]}的推荐系统，支持对用户情绪^[15]、歌词文本或语音特征^{错误!未找到引用源。}的识别与推送，但缺乏对用户上传音频文件的直接分析能力，检索与推荐方式多为简单的标签匹配或协同过滤，精度与个性化程度较低。此外，现有国内系统还存在明显的共性问题：部分系统功能冗余、界面复杂，操作流程不符合普通用户的使用习惯；部分系统部署成本高，依赖专业服务器与技术维护团队，难以在小范围或校园场景中落地推广；少数系统存在数据安全隐患，用户上传音频的审核机制与隐私保护措施尚不完善。

综合来看，现有文献主要有三类特点。第一，算法研究成果丰富，围绕音频特征、情感分析、协同过滤、知识图谱^[17]和深度学习提出了大量模型。第二，系统研究开始从单一推荐算法向全栈化产品实现延伸，逐步关注用户界面、后台管理与平台运营问题。第三，关于“用户上传音频后直接触发内容分析并生成推荐”的工程化系统研究仍相对较少，尤其是在同时整合收藏、社区反馈、AI 配置管理方面还缺少较完整的实现样本。因此，结合实际项目构建一套支持上传分析、推荐闭环、社区互动和后台管理的个性化音乐推荐系统，具有较强的研究价值。

1.3 主要研究内容

本文围绕当前项目已经实现的个性化音乐推荐系统展开研究，主要工作包括以下几个方面：

在需求分析层面，系统梳理数字音乐场景下用户在音频上传分析、音乐浏览播放、收藏管理、社区互动及推荐反馈等方面的业务需求，并结合管理员在曲库维护、用户管理、社区内容审核及 AI 配置管理等方面的运维需求，构建清晰的功能边界与角色边界。

在系统架构层面，采用 Spring Boot 构建后端服务模块，结合 Vue 3 实现前端响应式界面，通过前后端分离架构解决了系统模块间依赖过紧、独立部署与维护困难的问题。针对结构化数据持久化与热点数据访问延迟的问题，设计了 MySQL 与 Redis 的多数据源协同机制。

在核心功能实现层面，借助对 Spring AI 的 Alibaba DashScope 的音频特征的深度挖掘，既能对歌曲的节奏、音色等多维的特征向量的提取，将其作为推荐的主要依据，构建了基于内容的轻量级的推荐引擎，并将其对接了异步的处理流程（用户的上传触发了特征的提取、相似度的计算完成了歌曲的召回与排序、生成了个性化的推荐列表及分析报告等），同时又将用户的历史的反馈都做了充分的融合，从而将推荐的结果都做了闭环的优化，有

效的就既解决了推荐系统的冷启动阶段的准确率低的问题，也解决了对用户的偏好都捕捉不充分的问题等。

在系统测试与优化层面，依托于对系统的充分的测试与优化不仅将对系统的各个功能的模拟验证了其能否真正地正常的运行同时也对用户端和管理端的关键的业务流程都作了充分的分析，从而对系统的功能的完整性、各个模块之间的交互的连贯性及后台的管理的协同性等方面都给出了比较全面的评估，并为系统的后续的优化提供了比较可靠的方向和依据。

1.4 论文组织结构

本论文共分为 7 章，各章节核心内容安排如下：

第 1 章为引言，阐述本课题的研究背景与意义；综述国内外的研究现状，指出现有研究的局限性；概括本文的核心研究内容与论文组织结构，为后续章节的展开奠定基础。

第 2 章为相关技术，详细介绍系统开发所采用的核心技术栈：Spring Boot、MyBatis、JWT、Vue 3、Axios、MySQL、Redis、MinIO、Spring AI Alibaba DashScope 调用大语言模型实现音频特征提取与个性化推荐生成。说明各技术选型的原因。

第 3 章为系统分析，从可行性、需求、用例建模及类模型设计四个方面展开，明确开发可行性、核心需求与主要类间关系。

第 4 章为系统设计，完成总体架构、功能模块、活动图、时序图、类关系、页面跳转及数据库设计。

第 5 章为系统实现，介绍用户端与管理端核心模块的实现效果、关键代码与数据处理流程。

第 6 章为系统测试，说明测试目的、环境与方法，设计功能测试用例并验证系统与需求的一致性。

第 7 章为总结与展望，总结研发中遇到的问题与解决方法，分析系统不足，提出后续改进方向。

最后，通过参考文献与致谢对论文内容进行补充，参考文献列出论文撰写过程中引用的国内外相关文献（如音频特征提取、机器学习推荐、情感分析等方向的论文及学位论文），致谢部分阐述研发过程中的收获与感谢。

2 相关技术介绍

2.1 Spring Boot 后端开发框架

本系统后端采用 Spring Boot 3.2.5 构建。Spring Boot 具有约定优于配置、依赖整合能力强、生态成熟等特点，适合用于搭建中小型业务系统与前后端分离应用。在本系统中，Spring Boot 为控制器、服务层、配置管理、文件上传、邮件服务、Redis 连接和 Spring AI 调用提供了统一的运行基础，使得用户认证、音乐管理、上传分析和后台管理模块能够在同一工程中保持清晰分层。

2.2 Vue 3 与 Vite 前端技术

前端部分采用 Vue 3、Vite、Pinia、Vue Router 与 Element Plus 组合构建。Vue 3 提供了组合式 API 和响应式数据能力，便于构建音乐卡片、播放器、社区评论、AI 监控等复杂交互组件；Vite 则提供了快速开发和构建支持。Pinia 用于维护用户登录状态、当前播放音乐、播放列表与播放模式等全局数据，Vue Router 则统一管理用户端与管理端路由。Element Plus 为表单、表格、弹窗、分页和图表容器提供了成熟的界面组件支持。

2.3 MyBatis 与 MySQL 数据持久化

系统采用 MyBatis 作为数据访问层框架，使用 MySQL 作为关系型数据库。MyBatis 通过 Mapper 接口与 XML 映射文件配合实现 SQL 的细粒度控制，这使得本系统能够针对音乐列表分页、收藏夹查询、后台条件筛选等场景编写更灵活的查询语句。

MySQL 主要承担用户、音乐、上传记录、推荐记录、反馈评论、公告、站点内容、AI 配置、报告模板和个性化标签等结构化数据的持久化存储任务。通过主键、外键和索引设计，系统能够在保证数据一致性的同时支持较高频率的查询和更新操作。

2.4 Redis 缓存与状态管理

Redis 在本系统中的主要用途有两个。第一，用于保存用户播放列表和播放状态，包括当前播放曲目、播放模式、播放进度和是否正在播放等信息，从而实现用户重新登录后播放状态的恢复。第二，Redis 可作为热点数据与统计数据的辅助存储基础，为后续扩展用户行为分析和排行榜功能提供支持。

2.5 MinIO 对象存储

系统中的音频文件、封面图片、头像资源和站点图片统一存储在 MinIO 中。MinIO 具备部署简单、接口兼容 S3、适合本地或私有化部署等特点。由于系统需要支持用户上传音频、管理员上传曲库音频和封面、用户上传头像以及后台站点内容上传图片，采用对象存储可以避免大文件直接存储在数据库中，提升系统扩展性和资源管理效率。

在实现上，系统通过文件服务统一封装了资源保存和删除逻辑，不同模块只需要传入文件类型和目标目录即可完成上传处理，从而减少重复代码。

2.6 JWT 认证与邮件验证码机制

系统在身份认证上使用 JWT 令牌机制。用户登录成功后，后台生成包含用户 ID、用户名和角色信息的 Token，前端在后续请求中携带该 Token 完成身份校验。拦截器负责解析 Token 并将用户信息注入请求上下文，从而实现普通用户接口与管理员接口的统一权限控制。

为了支持注册校验和找回密码功能，系统还引入了邮件验证码服务。用户在注册和重置密码场景下，需要通过邮箱接收验证码并完成校验。该机制在提升账号安全性的同时，也增强了系统的完整性和可用性。

2.7 Spring AI Alibaba DashScope

本系统的推荐分析能力基于 Spring AI Alibaba DashScope 实现。系统通过 ChatClient 调用大模型服务，对用户上传的音频文件进行结构化分析，生成风格、情绪、节奏、乐器、氛围等字段描述，并进一步构造 5 维特征向量用于音乐相似度计算。与传统仅依赖元数据或用户行为的推荐方式相比，这种方式能够更直接地利用音频内容本身开展推荐。

同时，系统后台还提供了 AI 配置管理、AI 服务状态监控和报告模板管理能力，使得 AI 不只是一个黑盒推荐接口，而是成为一个可配置、可监测、可解释的业务模块。这一点与当前音乐平台中算法与产品深度融合的发展趋势一致。

2.8 本章小结

本章介绍了系统实现所依赖的关键技术栈。Spring Boot 提供了稳定的后端支撑，Vue 3 与 Vite 保证了前端交互效率，MyBatis 与 MySQL 完成结构化数据管理，Redis 负责状态缓存，MinIO 支撑大文件对象存储，JWT 与邮件验证码保障用户认证安全，Spring AI Alibaba DashScope 则负责音频分析和推荐生成。这些技术共同构成了本系统的实现基础。

3 系统分析

3.1 可行性分析

3.1.1 技术可行性

从技术角度看，本系统所采用的开发技术均较为成熟。Spring Boot、Vue 3、MyBatis、MySQL、Redis、MinIO 和 Element Plus 都具有完备的生态支持，能够满足毕业设计项目在前后端联动、对象存储、会话状态管理和后台管理方面的需求。AI 分析部分虽然具有一定挑战，但当前项目并未自行训练深度模型，而是通过 Spring AI 接入 DashScope 大模型，并结合音频特征向量构建轻量化相似度计算机制，因此整体技术路径是可实现的。

3.1.2 经济可行性

本系统主要依托开源技术栈完成实现，包括 Spring Boot、Vue 3、MyBatis、MySQL、Redis、MinIO 及 Spring AI Alibaba DashScope 等，不需要额外采购大型商业平台或专有软件授权。系统开发阶段的核心成本集中在功能实现、环境搭建与接口调试上，不会引入高额的商业软件采购费用。AI 调用虽然涉及接口成本，但在毕业设计阶段主要用于功能验证和实验分析，规模可控，因此总体经济成本处于可接受范围内。

3.1.3 社会可行性

数字音乐已经成为用户日常娱乐的重要组成部分，个性化推荐系统在提升内容发现效率、改善用户体验、增强平台粘性方面具有直接价值。同时，加入社区互动和反馈机制后，系统不仅服务于个人内容消费，也为用户之间的音乐交流提供了空间，具备一定的社会交互价值。从应用层面看，该系统适合作为校园项目、课程设计成果和音乐平台原型系统，因此具有较好的应用可行性。

3.2 需求分析

3.2.1 系统场景描述

本系统面向两类角色：普通用户和管理员。普通用户进入系统后，首先完成注册、登录和个人资料维护；随后可以浏览曲库、播放歌曲、收藏喜爱内容，并通过上传音频获取 AI 分析结果和推荐列表；在发现感兴趣的歌曲后，还可以进入社区页面查看动态、发表

评论、点赞评论并形成反馈闭环。管理员则通过后台完成用户状态维护、音乐资源上传与编辑、站点内容和公告发布、上传记录管理、社区反馈管理以及 AI 配置与监控等工作。

3.2.2 功能分析

本系统围绕用户端与管理端两方面进行服务，具体功能分布如下：

(1) 从用户端看，系统主要包含用户认证与个人资料、音乐浏览与播放、收藏夹、音频上传分析、推荐历史、社区互动和反馈等功能。用户可以通过首页搜索音乐，通过详情页查看封面、歌词和评论，并在播放过程中保存播放列表和播放状态。用户上传音频后，系统将保存上传记录、完成 AI 特征分析并生成推荐列表，同时允许用户对推荐结果和歌曲内容进行点赞、收藏、评论和反馈。

(2) 从管理端看，系统主要包含仪表盘统计、用户管理、音乐库管理、上传管理、反馈与社区管理、站点内容管理、通知公告管理和 AI 模块管理等功能。管理员可以维护音乐元数据和文件资源，查看和删除不合规社区内容，发布通知公告，调整站点轮播内容，监控 AI 服务状态并维护 AI 配置和分析报告模板。

3.2.3 角色分析

普通用户是系统的主要使用者，其核心目标是发现和享用音乐内容，并在推荐和社区功能中形成持续互动。管理员是平台维护者，其职责是保证用户数据、音乐内容、社区环境和 AI 模块的稳定运行与可控配置。系统通过角色字段和拦截器实现权限控制，使普通用户接口和管理员接口在访问边界上保持清晰分离。

3.3 用例建模

3.3.1 系统用例图

系统用例图分为普通用户用例图和管理员用例图两部分。普通用户主要围绕账号管理、音乐浏览播放、上传分析、收藏社区和反馈互动展开；管理员则围绕音乐资源管理、用户管理、社区管理、内容运营与 AI 管理展开。通过将用户端和管理端用例分别建模，可以更清晰地刻画系统在不同角色下的功能边界。

(1) 用户端用例图

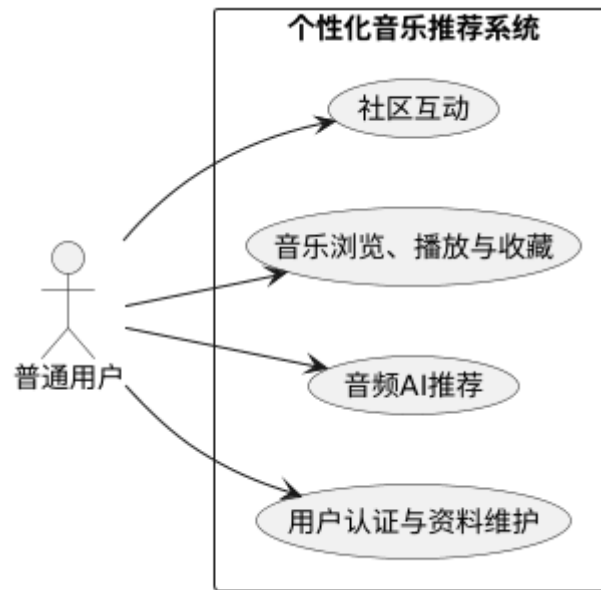


图 3.1 用户端用例图

普通用户作为系统的核心使用者，其主要用例围绕用户认证与资料维护、上传音频并生成 AI 推荐、音乐浏览播放与收藏、社区互动与反馈等核心活动展开。具体而言，用户首先完成账号注册、登录，系统完成身份认证与权限校验；登录成功后，可上传本地音频文件并使用 AI 特征分析，系统生成个性化推荐列表与分析报告；也可通过音乐主页对系统内的音乐资源进行浏览、播放与收藏，快速定位并管理个人喜爱的歌曲；此外，用户可在社区中或播放页面进行评论、回复、点赞及反馈互动。

除此之外，用户还可以对自身资料进行适当修改，通过查看历史推荐记录，找到过去曾推荐过的曲目，避免了多次重复的上传操作。

（2）管理员用例图

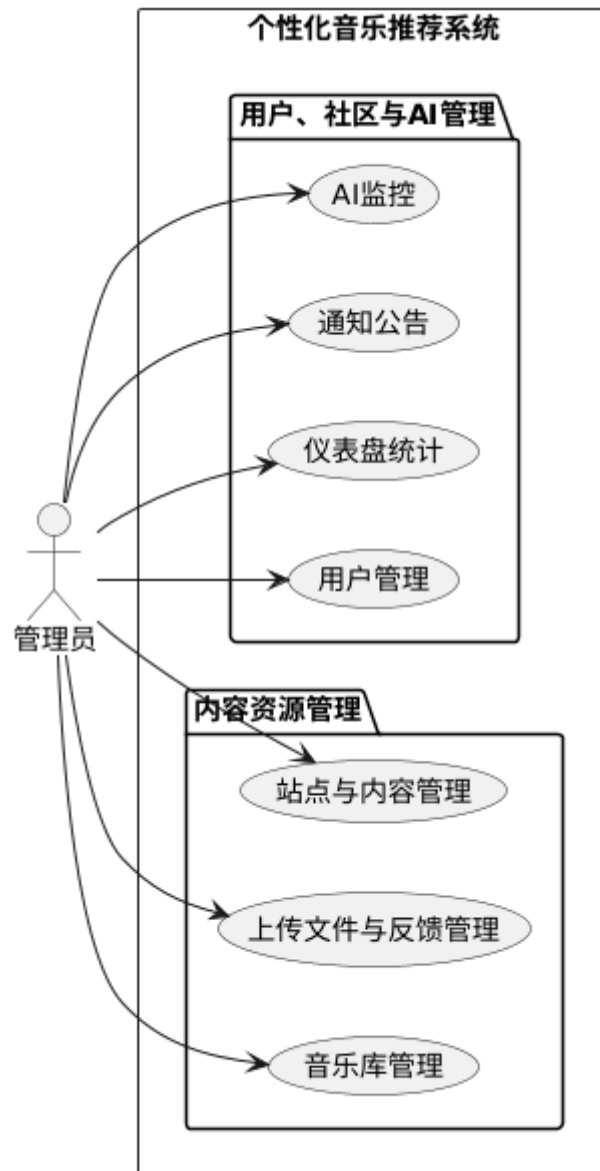


图 3.2 管理员用例图

系统管理员作为平台的管理者，其核心用例主要围绕用户管理、音乐库管理和社区内容管理等后台运维工作展开。在用户管理方面，管理员负责查看用户详情、调整权限、激活或删除账号，保障平台用户数据的时效性和安全性；在音乐库管理方面，管理员可以对歌曲进行新增、编辑、删除等操作，维护曲库的丰富性和准确性；在社区内容管理方面，管理员需要审核用户评论、设置首页轮播图与热门歌单，并发布系统通知公告，确保平台内容合规、信息透明。

此外，管理员还可以查看仪表盘统计中的系统访问量、推荐点击率等统计数据，以及监控 AI 推荐准确率与用户反馈聚合，为算法优化和平台运营提供数据支持。

3.3.2 用例规约

(1) 用户认证与资料维护用例规约主要用于描述用户登录注册以及修改账户内容时的活动详情，具体内容如下表 3.1 所示。

表 3.1 用户认证与资料维护

要素	含义与要求
用例编号: UC001	用例名称: 用户认证与资料维护
简要描述:	用户完成注册、登录、找回密码、修改资料、上传头像和修改密码等操作
用例角色:	普通用户
前置条件:	1. 用户已进入系统首页或认证页面; 2. 进行资料维护时用户已登录
后置条件:	1. 用例执行成功: 登录成功后系统返回 JWT 令牌并建立会话状态, 资料维护成功后数据库中的用户信息被更新 2. 用例执行失败: 系统返回对应的错误提示
基本流程:	1. 用户输入账号密码或注册信息; 2. 系统校验用户名、邮箱和验证码合法性; 3. 登录成功后生成 JWT 并返回用户基本信息; 4. 用户进入个人中心查看和修改昵称、头像、偏好等资料; 5. 系统保存变更结果并返回操作成功提示
备选流程:	E-1: 账号或密码错误; E-2: 注册邮箱已存在; E-3: 验证码错误或过期; E-4: 上传头像格式不合法或超过大小限制
业务规则:	1. 邮箱验证码需要在有效期内使用; 2. 用户密码采用 BCrypt 加密; 3. 管理员与普通用户登录入口分离;

(2) 音频 AI 推荐用例规约主要描述用户在上传音频时系统进行分析与反馈的操作，具体内容如下表 3.2 所示。

表 3.2 音频 AI 推荐

要素	含义与要求
用例编号: UC002	用例名称: 音频 AI 推荐
简要描述:	用户上传本地音频文件, 系统完成音频分析、特征提取、推荐理由生成和相似音乐推荐
用例角色:	普通用户
前置条件:	1. 用户已成功登录系统; 2. 本地存在合法音频文件; 3. 系统已配置可用的 AI 服务

要素	含义与要求
后置条件:	1. 用例执行成功: 上传记录被保存, 分析结果写入上传记录, 推荐记录被生成并可供查询 2. 用例执行失败: 系统返回对应的错误提示
基本流程:	1. 用户选择音频文件并上传; 2. 系统保存文件到对象存储并创建上传记录; 3. 用户触发分析请求; 4. 系统将上传状态更新为 ANALYZING; 5. AI 模块提取音频描述和特征向量; 6. 系统生成推荐理由并计算与曲库音乐的相似度; 7. 系统保存推荐记录; 8. 用户查看分析报告与推荐列表

表 3.2 (续) 音频 AI 推荐

要素	含义与要求
备选流程:	E-1: 文件为空或格式错误; E-2: 分析服务调用失败; E-3: 上传记录不存在或无权限访问; E-4: 曲库中无可匹配特征导致推荐为空
业务规则:	1. 仅登录用户可访问上传分析功能; 2. 上传记录只能由所属用户访问; 3. 推荐排序依据相似度降序

(3) 音乐浏览、播放与收藏用例规约是描述用户进入系统之后对音乐进行浏览与播放之类的操作, 详细内容如下表 3.3 所示。

表 3.3 音乐浏览、播放与收藏

要素	含义与要求
用例编号: UC003	用例名称: 音乐浏览、播放与收藏
简要描述:	用户在系统中搜索和浏览音乐, 查看详情并进行播放、收藏、保存播放列表和播放状态
用例角色:	普通用户
前置条件:	1. 用户已登录系统; 2. 曲库中存在可用音乐数据
后置条件:	1. 用例执行成功: 系统记录播放列表状态, 收藏操作写入收藏关系, 用户可以在收藏夹中再次查看已收藏音乐 2. 用例执行失败: 系统不保存任何信息, 返回对应的失败提示

要素	含义与要求
基本流程:	1. 用户进入首页或搜索页获取曲库列表; 2. 选择歌曲进入详情页; 3. 系统展示封面、歌词和评论; 4. 用户播放歌曲并切换播放模式; 5. 系统将播放列表与播放状态写入 Redis; 6. 用户点击收藏按钮将歌曲加入个人收藏夹; 7. 用户在收藏页查看已收藏歌曲
备选流程:	- E-1: 音乐不存在或已下架; - E-2: 音频文件加载失败; - E-3: 收藏已存在时取消收藏; - E-4: 播放列表为空时无法切歌
业务规则:	1. 收藏关系按用户隔离; 2. 播放模式支持顺序播放、单曲循环和随机播放; 3. 播放列表和播放状态持久化到 Redis; 4. 用户只能访问已上架音乐

(4) 社区互动用例规约是描述用户在歌曲评论区部分进行互动的内容, 详细如下表 3.4 所示。

表 3.4 社区互动

要素	含义与要求
用例编号: UC004	用例名称: 社区互动
简要描述:	用户围绕歌曲进行评论、回复、点赞、查看社区动态, 并提交推荐反馈
用例角色:	普通用户
前置条件:	1. 管理员已成功登录系统; 2. 目标歌曲存在; 3. 评论数据可访问
后置条件:	1. 用例执行成功: 评论、回复和点赞关系被保存; 社区动态列表更新; 反馈内容可供后台查看; 2. 用例执行失败: 系统不保存任何信息, 返回对应的失败提示
基本流程:	1. 用户进入歌曲详情页评论区或社区页; 2. 系统加载歌曲评论和社区动态; 3. 用户发表评论或回复评论; 4. 用户点赞评论或对推荐结果提交反馈; 5. 系统保存反馈记录并更新互动状态; 6. 用户在社区页继续浏览动态
备选流程:	- E-1: 评论内容为空; - E-2: 回复目标不存在; - E-3: 重复点赞时执行取消点赞; - E-4: 删除评论时权限不足

要素	含义与要求
业务规则:	1.评论点赞不能对自己的评论进行; 2.收藏、点赞、评论和反馈统一进入反馈体系; 3.管理员可对社区内容进行查看和删除

(5) 内容资源管理用例规约是管理员在维护音乐库, 站点轮播等内容进行的操作, 用于保证平台内容资源完整, 详细如下表 3.5 所示。

表 3.5 内容资源管理

要素	含义与要求
用例编号: UC005	用例名称: 内容资源管理
简要描述:	管理员维护音乐库、站点轮播内容和通知公告, 保证平台内容资源的完整性与可用性
用例角色:	管理员
前置条件:	1. 管理员已成功登录系统; 3. 管理员具有管理权限
后置条件:	1. 用例执行成功: 音乐资源、站点内容或公告信息被新增、更新、发布或删除; 2. 用例执行失败: 系统不修改用户状态, 返回对应的失败提示
基本流程:	1. 管理员进入音乐管理或内容管理页面; 2. 上传音频文件、封面图片或站点图片; 3. 填写标题、歌手、分类、链接、状态等信息; 4. 系统将文件保存到对象存储; 5. 系统将元数据保存到数据库; 6. 前台首页和曲库页面按最新内容进行展示
备选流程:	E-1: 文件格式或大小不合法; E-2: 图片上传失败; E-3: 数据库更新失败; E-4: 删除内容时关联资源清理异常

表 3.5 (续) 内容资源管理

要素	含义与要求
业务规则:	1. 管理员新增音乐时音频文件为必填 2. 站点内容支持轮播图、横幅和热点内容等类型; 3. 资源删除后应尽量同步删除对象存储文件

(6) 用户、社区与 AI 管理用例规约是描述管理员对于用户、社区、AI 内容的统一管理, 详细内容如下表 3.6 所示。

表 3.6 用户、社区与 AI 管理

要素	含义与要求
用例编号: UC006	用例名称: 用户、社区与 AI 管理

要素	含义与要求
简要描述:	管理员对用户状态、社区反馈、上传记录、AI 配置、AI 监控和报告模板进行统一管理
用例角色:	管理员
前置条件:	1. 管理员已成功登录后台系统
后置条件:	1. 用例执行成功: 用户状态、社区内容或 AI 配置被更新, 监控数据可被查看;
基本流程:	2. 用例执行失败: 系统保持原有状态, 返回对应的失败提示
	1. 管理员进入后台;
	2. 查看用户列表并修改用户状态;
	3. 查看上传记录和反馈列表;
	4. 删除不合规社区内容;
	5. 查看 AI 监控统计;
	6. 新增、编辑、测试或切换 AI 配置;
备选流程:	E-1: 目标用户不存在;
	E-2: 删除社区内容失败;
	E-3: AI 配置测试不通过;
	E-4: 删除默认模板或当前启用配置时系统拒绝执行
业务规则:	1. 被禁用用户不能继续正常使用核心业务;
	2. 激活 AI 配置时应取消其他配置的激活状态;
	3. 社区管理复用反馈管理能力实现

3.4 系统主要类模型

本系统共包含 11 个核心类: User (用户)、UserPersonalTag (用户个性化标签)、Music (音乐)、UploadRecord (上传记录)、RecommendRecord (推荐记录)、Feedback (反馈)、CommentLike (评论点赞)、Notice (公告)、SiteContent (站点内容)、AIConfig (AI 配置) 和 ReportTemplate (报告模板)。具体详情如图 3.3 所示。

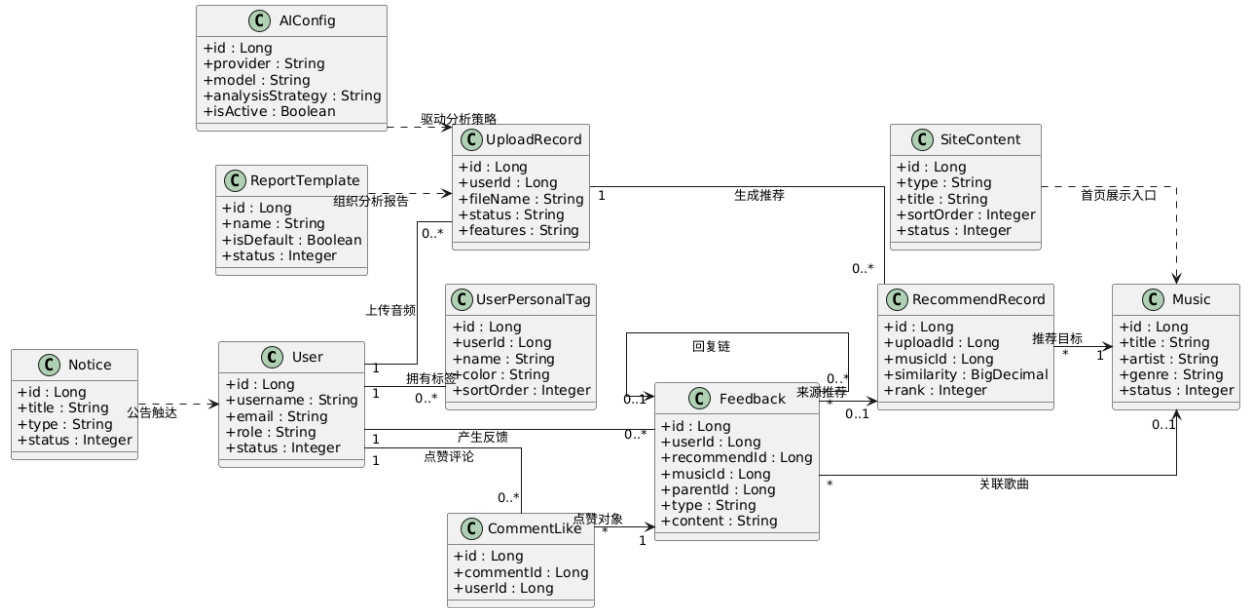


图 3.3 系统主要类模型

3.5 本章小结

本章从技术、经济和社会三方面论证了系统的可行性，明确了普通用户和管理员两类角色的功能需求，并通过用例图与 6 个核心用例规约进行了详细描述，最后给出了 11 个核心类的系统类关系图。

4 系统设计

4.1 系统总体结构

系统整体采用前后端分离架构。前端基于 Vue 3 构建用户端与管理端统一应用，后端基于 Spring Boot 实现控制层、服务层、数据访问层与基础支撑层。用户端通过浏览器访问首页、上传分析、收藏页、社区页和个人中心等页面；管理端通过后台页面完成用户、音乐、上传、反馈、站点内容、公告和 AI 模块的维护。后端再通过 MyBatis 访问 MySQL，通过 Redis 保存播放状态，通过 MinIO 保存文件资源，通过 Spring AI 调用 DashScope 模型完成音频分析。

为了更直观地说明前后端与基础支撑组件之间的协作关系，本文将系统总体结构划分为用户端、管理端、Spring Boot 后端以及底层存储与 AI 支撑层四个部分。用户端与管理端分别承载不同角色的页面访问入口；Spring Boot 后端作为业务处理核心，对外暴露统一接口；MySQL、Redis、MinIO 和 DashScope 则分别承担结构化数据存储、状态缓存、对象存储和智能分析能力。系统总体架构图代码如下：

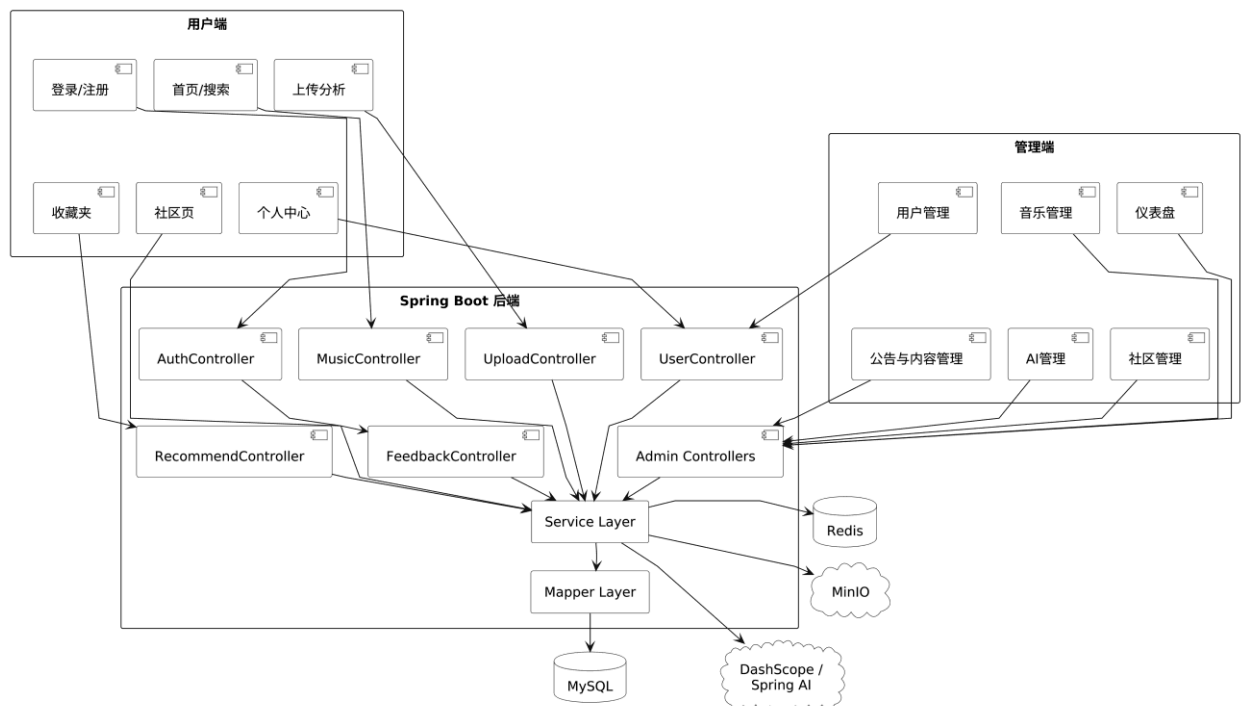


图 4.1 系统总体架构图

4.2 系统主要功能模块

本节针对系统核心用例进行活动图与时序图设计。活动图描述用例内部的处理流程和控制流，使用分支判断（菱形）表示条件分支，使用分叉与汇合（同步条）表示并行处理，清晰展示业务流程的各个环节；时序图描述同一用例中各对象之间的交互顺序，从左到右排列参与者，从上到下展示消息传递顺序，与活动图一一对应。活动图侧重流程控制，展示业务逻辑的判断与跳转；时序图侧重对象交互，展示各模块间的调用关系与数据传递。

4.2.1 系统活动图

本系统中对于用户认证与资料维护的实现由活动图体现了用户从注册、登录到进入个人中心并修改资料的处理逻辑，体现了验证码校验、JWT 令牌签发、头像上传与资料更新等关键步骤。详情如图 4.2 所示。

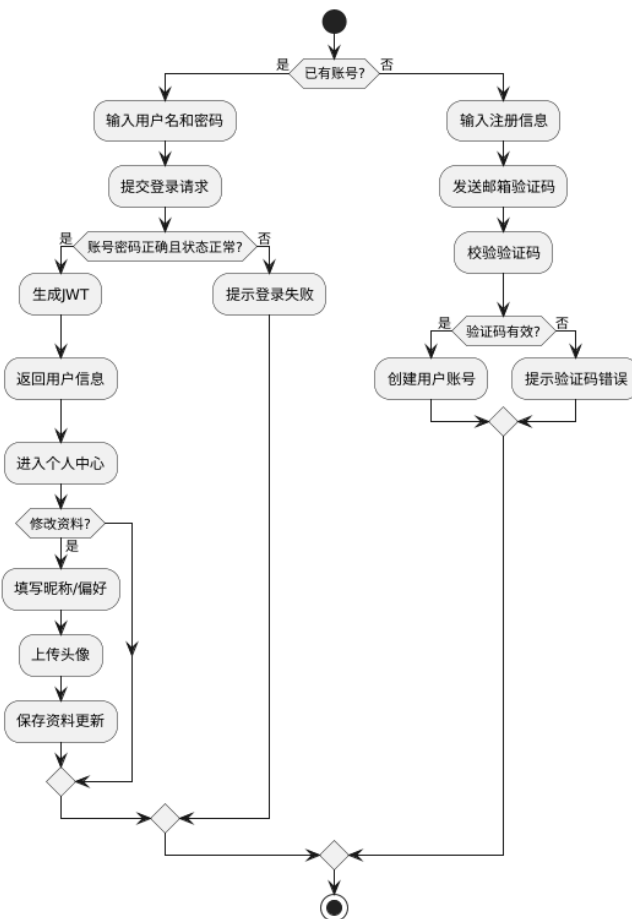


图 4.2 用户认证与资料维护活动图

本系统中对于音频 AI 推荐的实现由活动图体现了用户从上传音频到展示分析结果与推荐列表的处理逻辑，体现了创建上传记录、触发 AI 分析、生成特征向量、计算相似度并写入推荐记录等关键步骤。详情如图 4.3 所示。

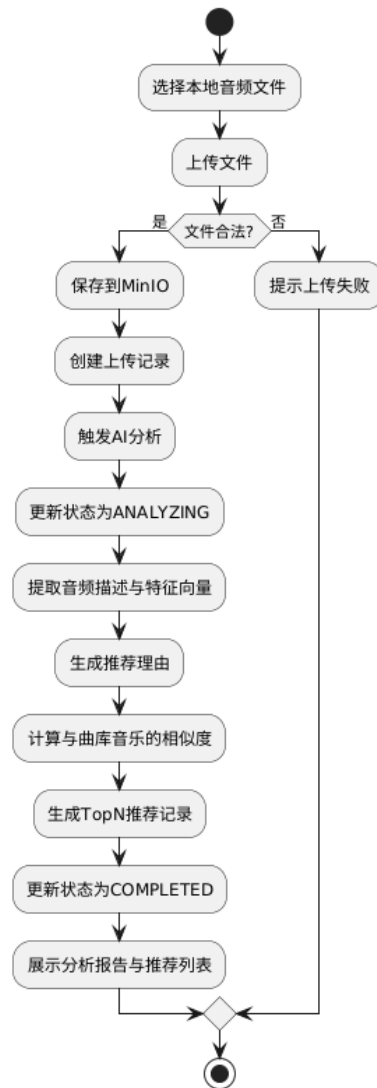


图 4.3 音频 AI 推荐活动图

本系统中对于音乐浏览播放与收藏的实现由活动图体现了用户从检索音乐到执行收藏操作的处理逻辑，体现了查看详情、开始播放、保存播放状态及执行收藏操作等关键步骤。详情如图 4.4 所示。

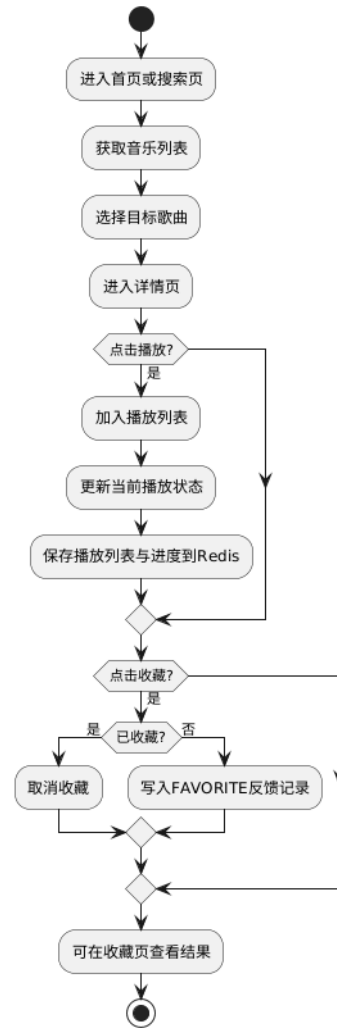


图 4.4 音乐浏览播放与收藏活动图

本系统中对于社区互动与反馈的实现由时序图体现了用户从查询评论列表到提交评论、回复、点赞及聚合社区动态的处理逻辑，体现了评论列表查询、评论提交、回复、点赞和社区动态聚合等关键步骤。详情如图 4.5 所示。

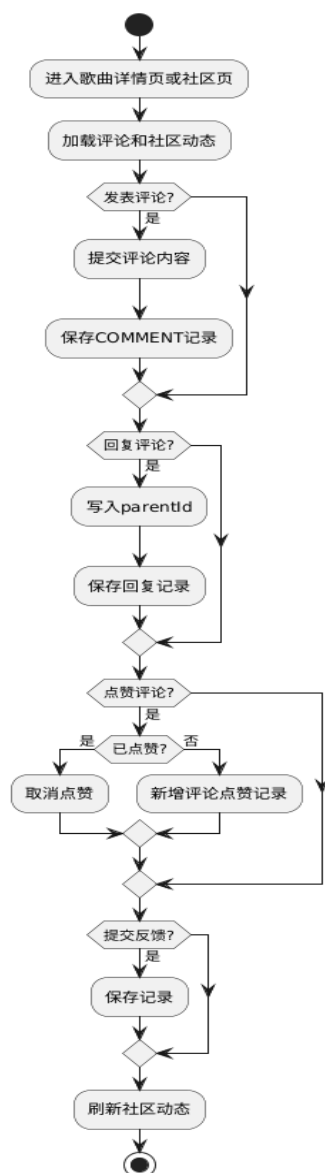


图 4.5 社区互动活动图

本系统中对于内容资源管理的实现由活动图体现了管理员从上传音乐文件、封面和站点内容图片到维护音乐元数据、公告信息和轮播内容的处理逻辑，体现了上传音乐文件、封面及站点内容图片、维护音乐元数据、公告信息和轮播内容等关键步骤。详情如图 4.6 所示。

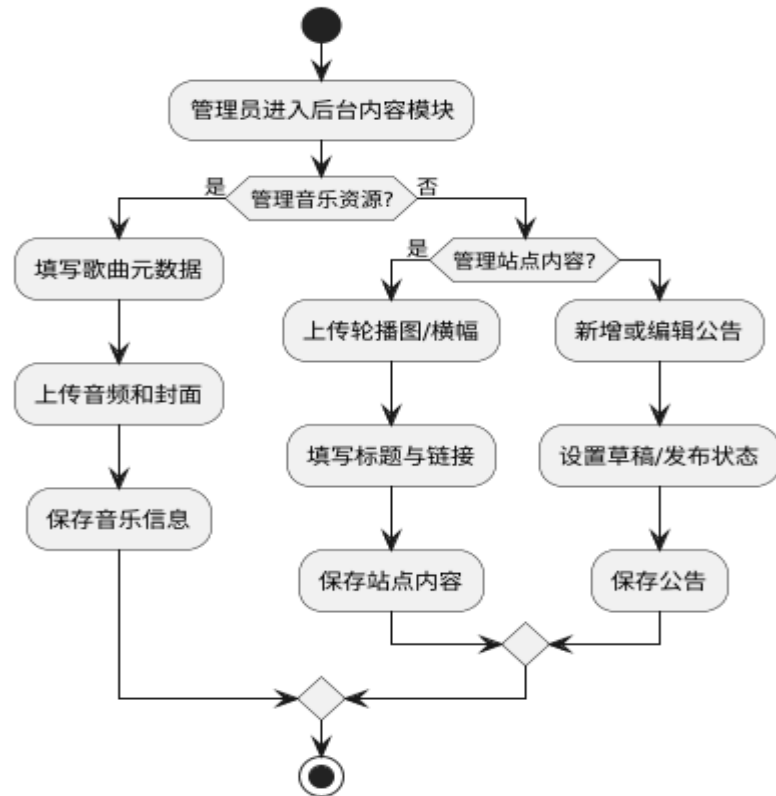


图 4.6 内容资源管理活动图

本系统中对于用户、社区与 AI 管理的实现由活动图体现了管理员从管理用户状态、社区反馈到管理 AI 配置的处理逻辑，体现了管理用户状态、社区反馈和 AI 配置等关键步骤。详情如图 4.7 所示。

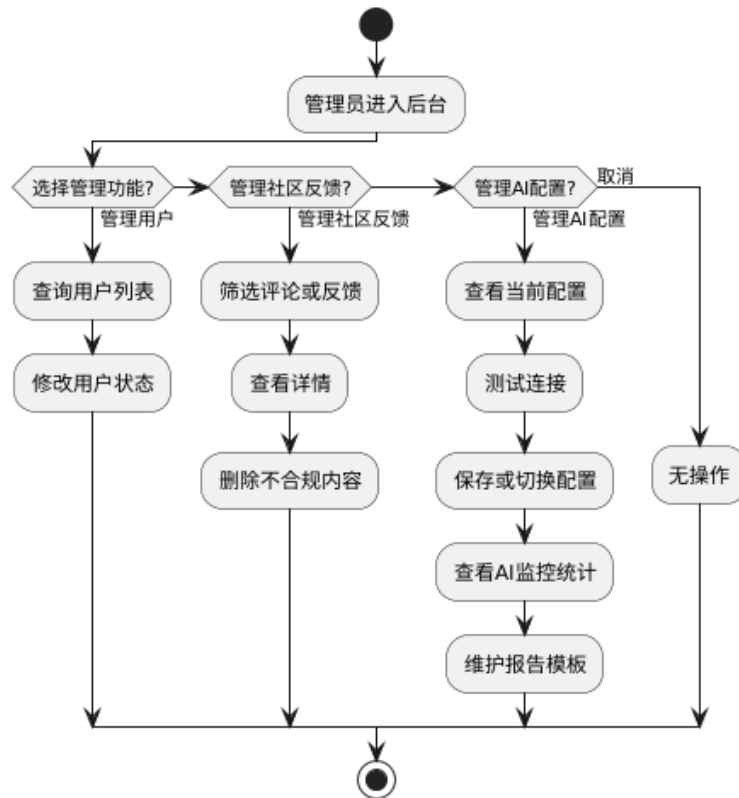


图 4.7 用户、社区与 AI 管理活动图

4.2.2 系统时序图设计

用户认证与资料维护时序图展示了用户、前端页面、认证控制器（AuthController）、用户服务（UserService）、验证码服务（VerificationCodeService）以及 MySQL 数据库之间的交互关系。整个时序图清晰地反映了注册时的验证码前置校验、登录后的令牌生成与传递、以及资料更新时对令牌的依赖。详情如图 4.8 所示。

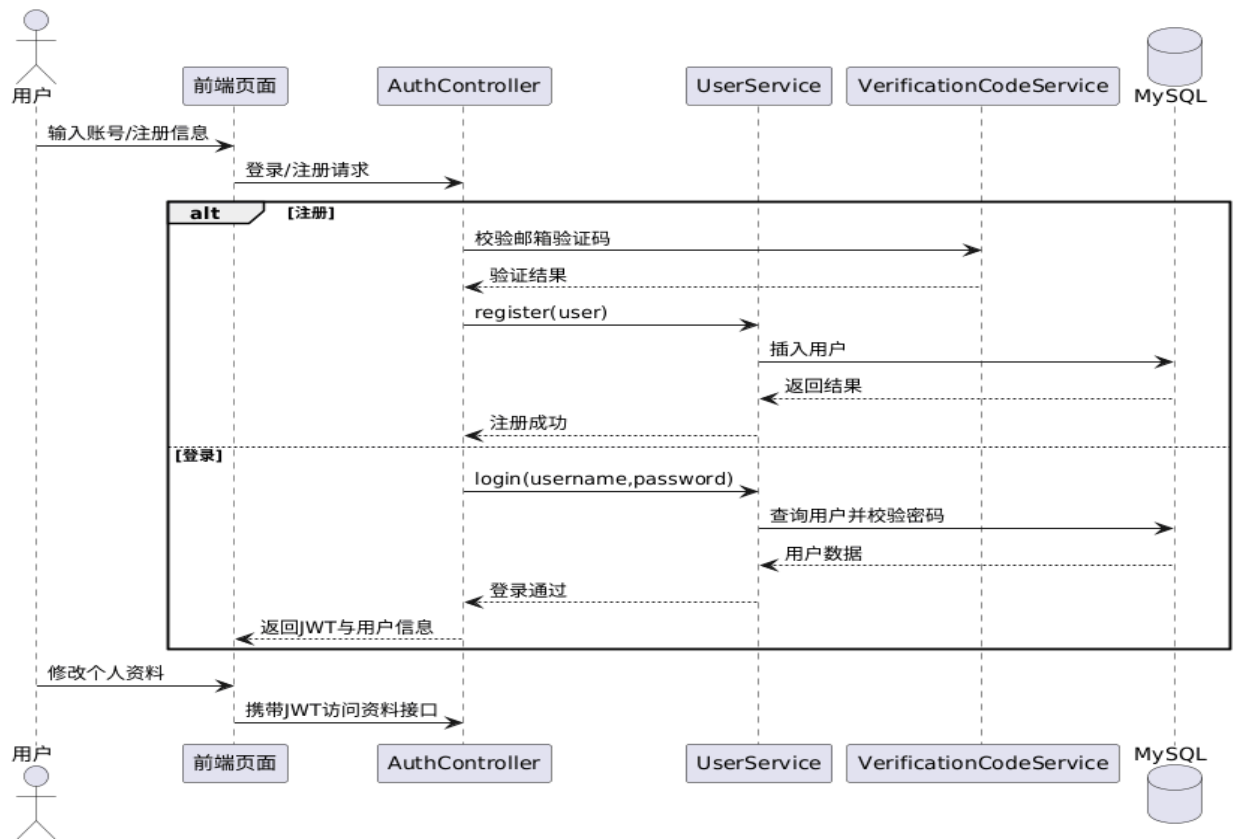


图 4.8 用户认证与资料维护时序图

音频推荐时序图是系统最关键的智能处理链路，整个时序图清晰地反映了音频文件的云端存储、异步特征提取与推荐生成的完整链路，体现了系统在音频内容理解与个性化推荐方面的核心智能化设计。详情如图 4.9 所示。

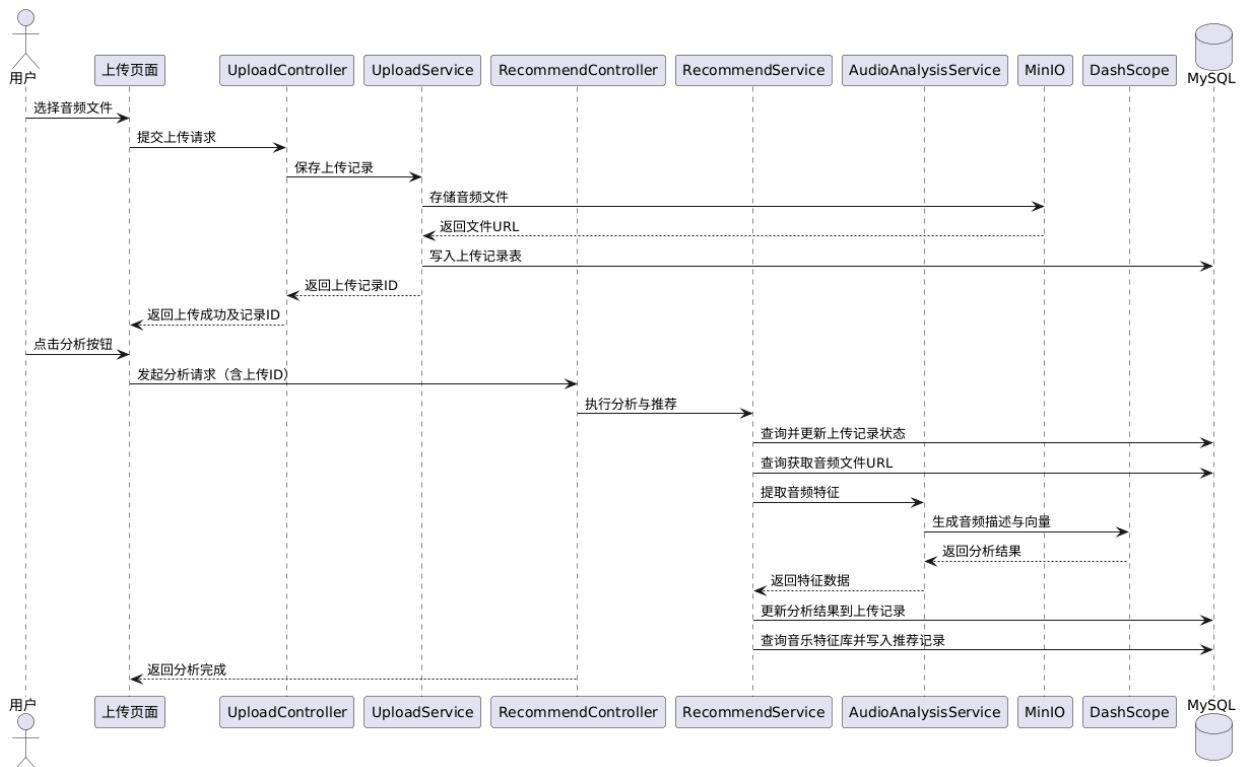


图 4.9 音频 AI 推荐时序图

以下为音乐浏览播放与收藏的时序图，体现了用户搜索或浏览音乐，系统查询并返回音乐列表；随后用户点击播放，系统保存播放列表与播放状态至 Redis；最后用户点击收藏，系统新增或删除收藏记录并返回收藏状态。详情如图 4.10 所示。

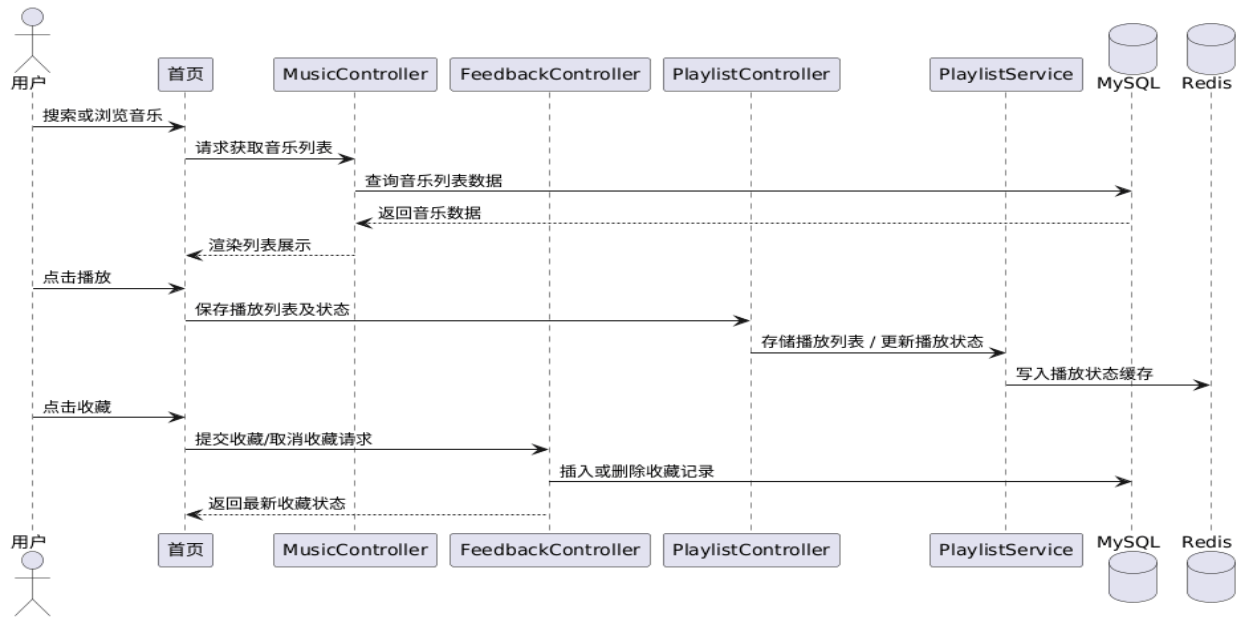


图 4.10 音乐浏览播放与收藏时序图

以下为社区互动的时序图，用户具体操作为：用户可以通过关联音乐信息查询社区动态并返回动态列表；随后用户发表评论或回复，系统写入评论记录；最后用户点赞评论，系统新增或删除点赞记录并返回点赞状态。详情如图 4.11 所示。

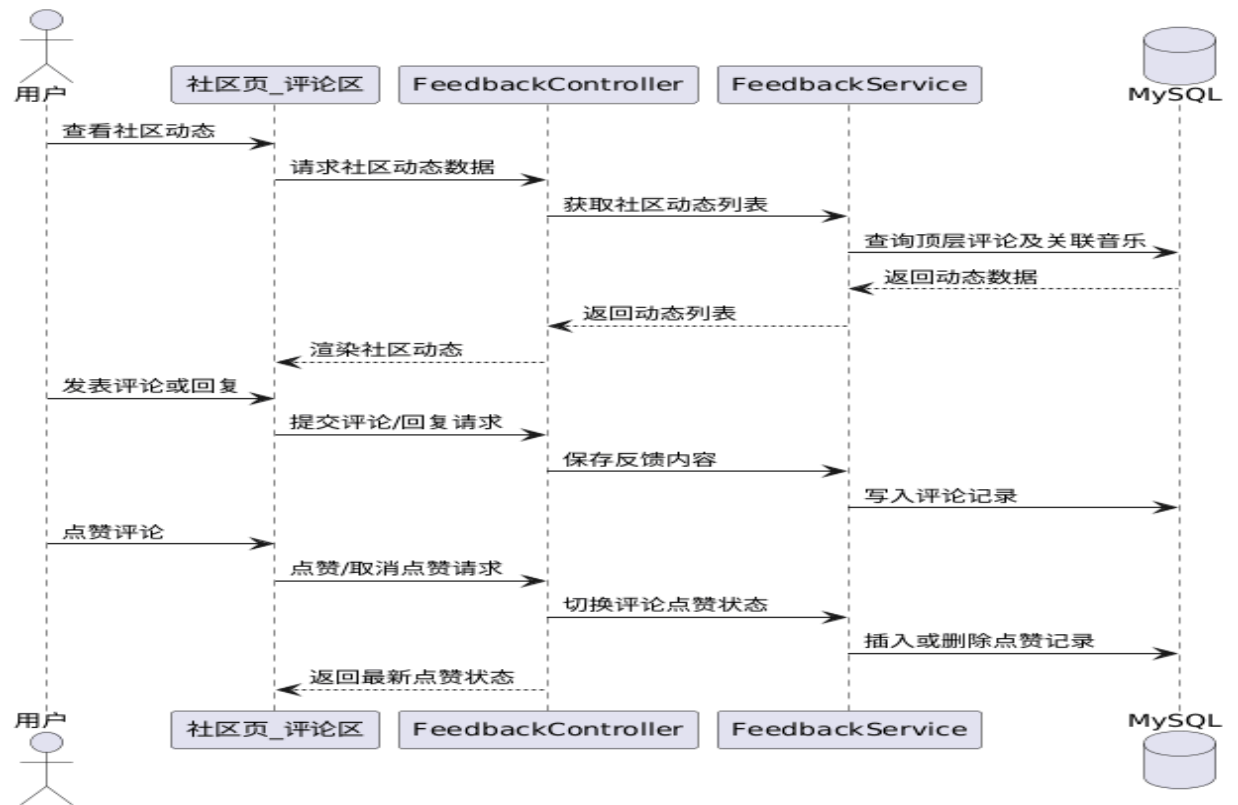


图 4.11 社区互动时序图

以下为管理员进行内容资源操作的时序图，管理员选择不同的操作进行不同的管理：若为音乐资源管理，则依次保存音频至 MinIO 并保存音乐数据至 MySQL；若为站点内容管理，则依次上传图片至 MinIO 并保存站点内容至 MySQL；若为公告管理，则直接保存公告至 MySQL。详情如图 4.12 所示。

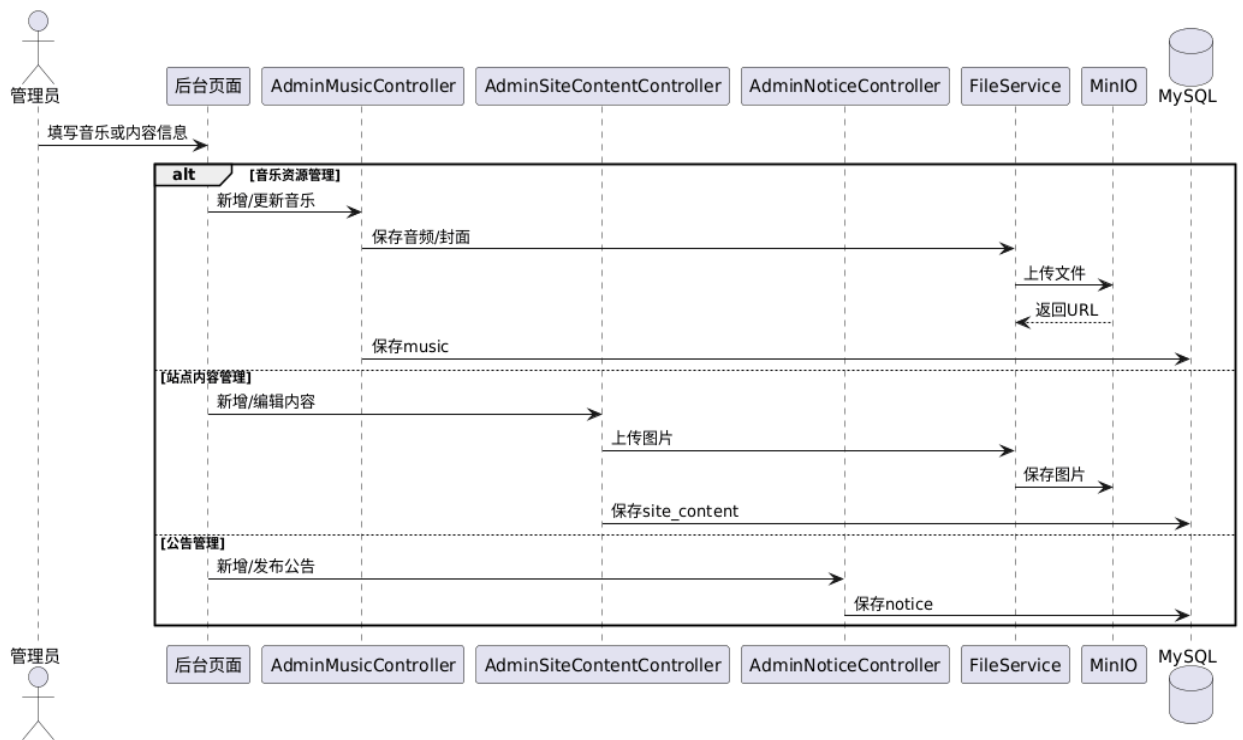


图 4.12 内容资源管理时序图

本系统中对于用户、社区与 AI 管理的实现由时序图体现了管理员从查询信息到完成各项管理的处理逻辑，依次体现了修改用户状态、查看或删除社区反馈、保存 AI 配置、查看 AI 监控并聚合数据等步骤。详情如图 4.13 所示。

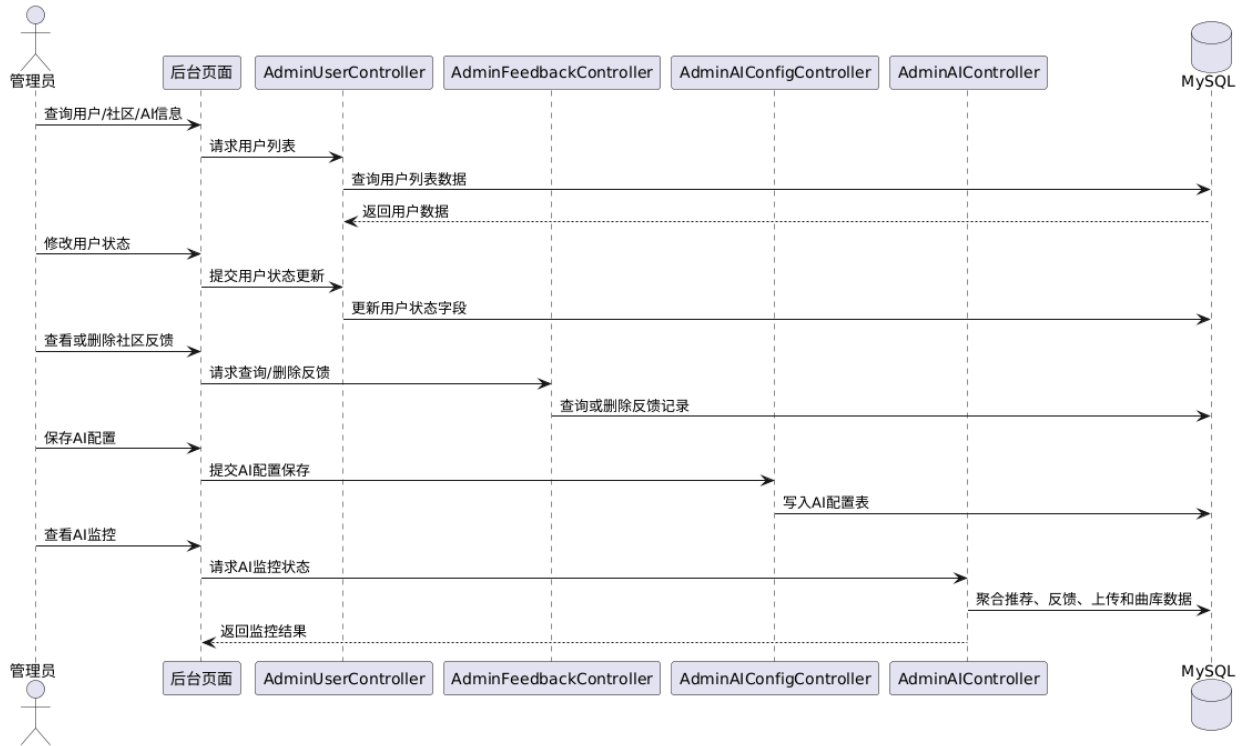


图 4.13 用户、社区与 AI 管理时序图

4.3 系统主要界面架构

4.3.1 主要系统类图

本系统在代码实现上遵循典型的 Controller、Service、Mapper 分层结构。Controller 负责接受请求并完成参数校验，Service 负责承载业务逻辑，Mapper 负责数据库持久化访问。对外部支撑组件而言，文件服务封装了 MinIO 访问逻辑，AI 分析服务封装了音频分析、特征向量生成和相似度计算逻辑，播放列表服务封装了 Redis 状态存储逻辑。

详细类图如图 4.14 所示：

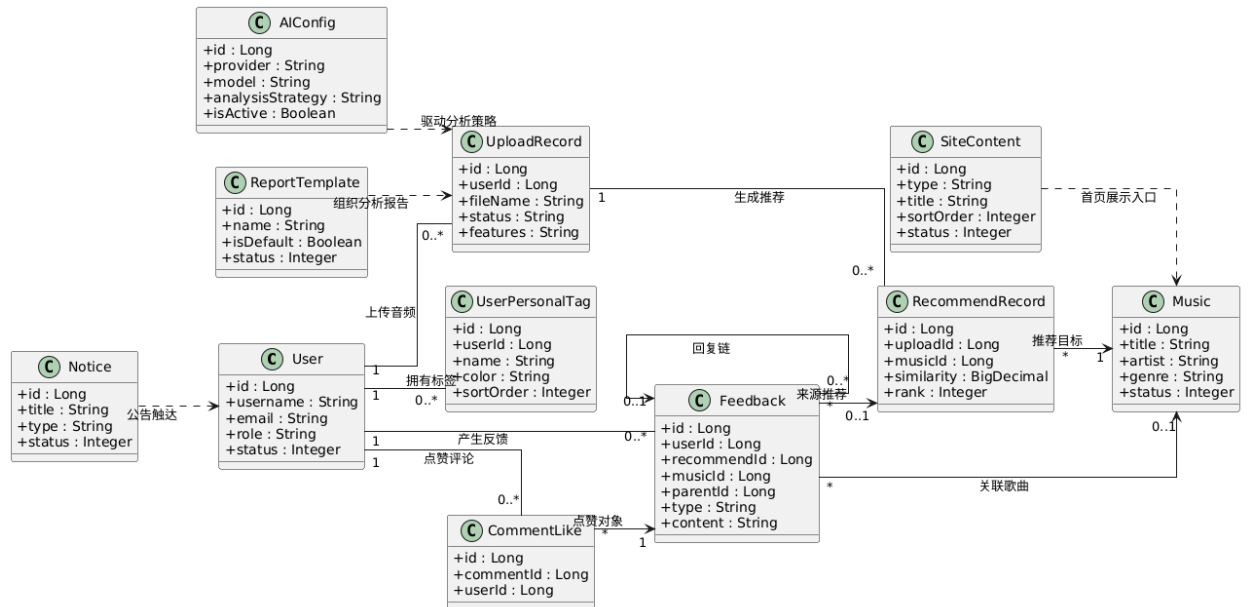


图 4.14 系统主要实体类图

4.3.2 系统页面关系

系统采用前后端分离的双端页面结构设计，将普通用户与管理员后台进行组织，从而降低不同角色在交互目标和页面权限上的耦合度。页面关系可概括为“独立登录入口 + 主框架页面 + 业务功能页面”的组织方式。

用户端中普通用户完成账号注册、登录或找回密码后进入用户端首页，首页作为统一的页面容器，承载音乐主页、上传分析页、推荐历史页、收藏页、社区页、个人中心和歌曲详情页等子页面。其中，音乐主页负责展示曲库列表、热门推荐及搜索入口，支持用户按歌手、风格或关键词筛选音乐；当用户选择具体歌曲后，系统跳转至独立的歌曲详情页，完成音乐播放、歌词查看、评论浏览、收藏切换和点赞等连续操作。上传分析页既承担音频上传功能，也承担分析结果展示和推荐列表查看功能；推荐历史页用于回顾以往的上传记录和推荐结果，收藏页集中展示用户收藏的音乐并支持取消收藏；社区页用于展示用户动态、热门评论和评论互动，个人中心用于维护头像、昵称、偏好等个人资料。

管理端中管理员通过账号密码登录后进入后台主框架页面，可以访问仪表盘、用户管理、音乐管理、上传管理、反馈管理、社区管理、站点内容管理、公告管理和 AI 管理等页面。用户管理页面用于查询用户列表、修改用户状态或权限；音乐管理页面用于完成音乐的新增、编辑、上架/下架和删除等操作；上传管理页面用于查看用户上传记录和分析结果；反馈管理与社区管理页面用于查看和删除评论、回复及点赞记录；站点内容管理页面用于维护轮播图、热门歌单等首页内容；公告管理页面用于发布和编辑系统通知；AI 管理页面用于配置 AI 服务参数、测试连接及查看监控统计。

上述页面关系体现出用户端围绕“发现音乐—分析推荐—互动反馈”的主线展开，而管理端围绕“资源治理—社区治理—AI 管理”的主线展开。对应页面关系图代码如图 4.15 以及图 4.16 所示：

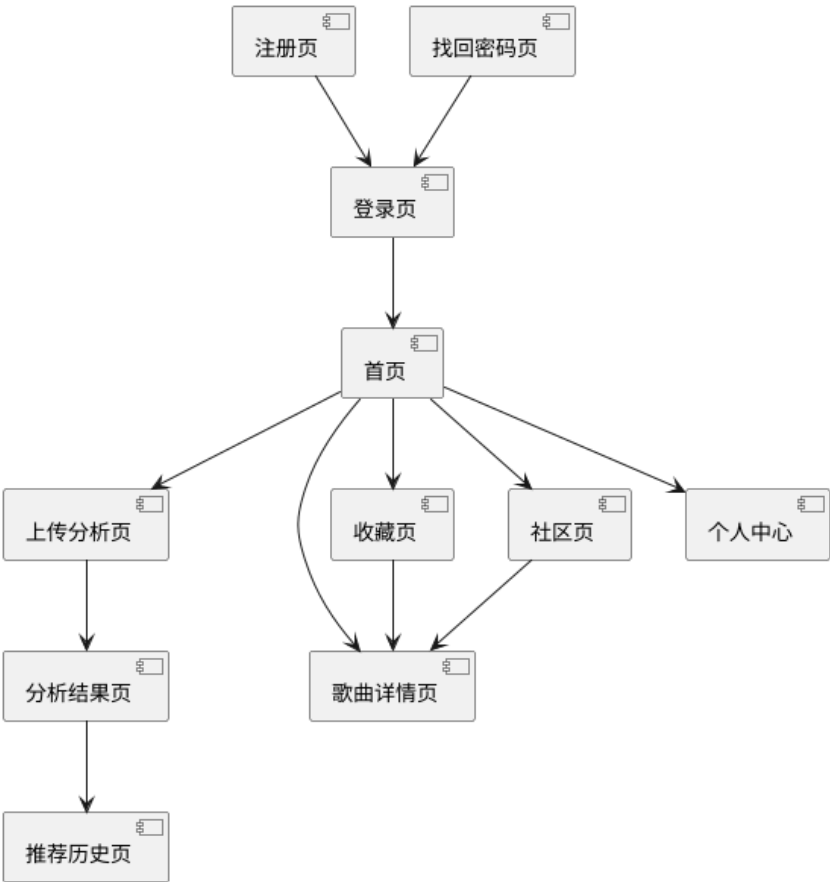


图 4.15 用户页面系统关系图

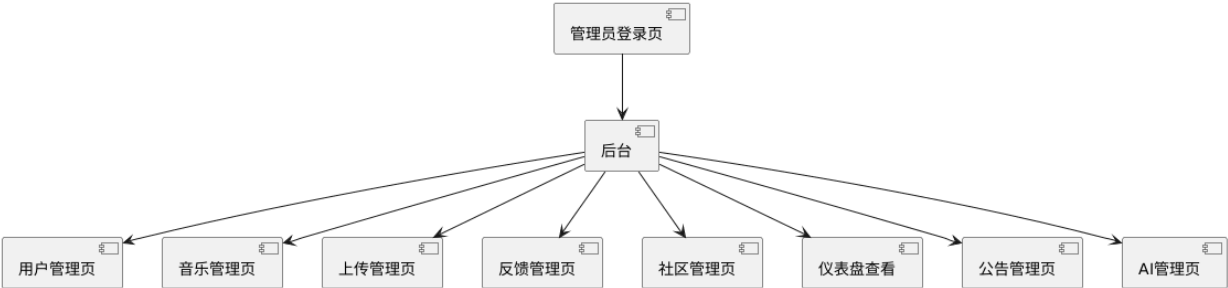


图 4.16 管理员页面系统关系图

4.4 数据库设计

4.4.1 数据库关系图

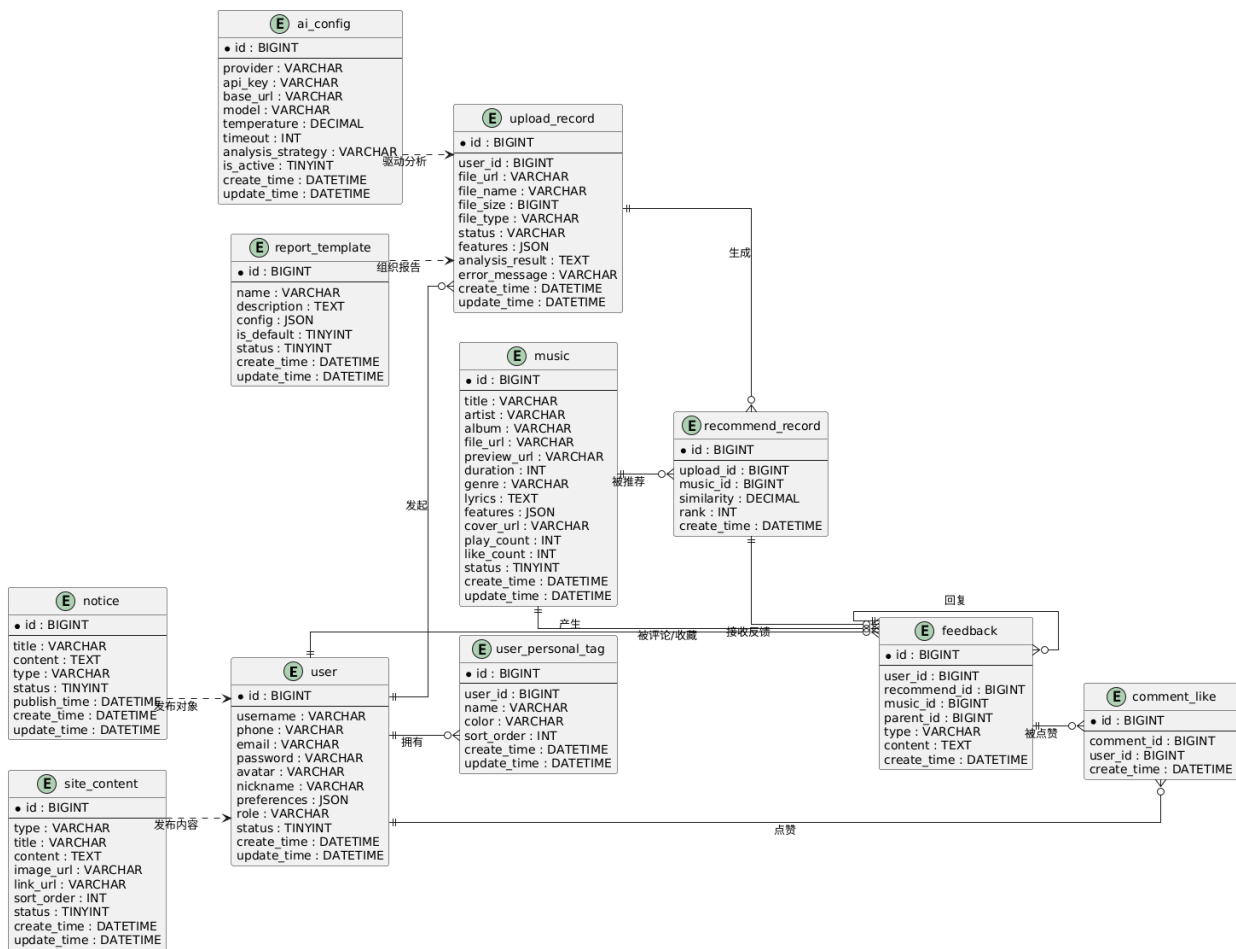


图 4.17 数据库关系图

4.4.2 数据库物理设计

系统核心数据库表包括用户表、音乐表、上传记录表、推荐记录表、反馈及社区表、评论点赞表、AI 配置表、报告表和数据表。下面分别对各表进行详细设计说明。

(1) 用户表用于存储用户的基本信息和登录凭证。用于区分账户的身份，可以提供系统管理端的权限控制基础。该表为系统所有操作的前提，只有数据存储在表中，操作者才可以继续使用该系统。用户表信息详情如表 4.1 所示。

表 4.1 用户表

列名	数据类型	长度	是否为空	默认值	备注
表 4.1 (续) 用户表					
id	BIGINT		否	自增	用户主键 ID
username	VARCHAR	50	否	-	用户名
phone	VARCHAR	20	是	-	手机号

列名	数据类型	长度	是否为空	默认值	备注	(2) 音
email	VARCHAR	100	是	-	邮箱	乐表用
password	VARCHAR	255	否	-	加密密码	于存储
avatar	VARCHAR	500	是	-	头像 URL	系统中
nickname	VARCHAR	50	是	-	昵称	已有的
preferences	JSON	-	是	-	用户偏好标签 JSON 数据	音乐数
role	VARCHAR	20	是	'USER'	角色标识, USER 或 ADMIN	数据, 包括
status	TINYINT	-	是	1	状态标识, 1 为 正常, 0 为禁用	歌曲名
						称、试听
						地址、歌
						曲时长、

风格分类等信息, 歌曲信息由管理员直接管理, 用户前台可以通过音乐表信息听取音乐。

音乐表信息详情如表 4.2 所示。

表 4.2 音乐表

列名	数据类型	长度	是否为空	默认值	备注
id	BIGINT	-	否	自增	音乐主键 ID
title	VARCHAR	200	否	默认值	歌曲名称
play_count	INT	100	是	0	播放次数
artist	VARCHAR	100	是	-	歌手名称
album	VARCHAR	200	是	-	专辑名称
file_url	VARCHAR	500	否	-	音频文件 URL, 存储于 MinIO
preview_url	VARCHAR	500	是	-	试听片段 URL
duration	INT	-	是	-	歌曲时长, 单位 为秒
genre	VARCHAR	50	是	-	风格或流派
lyrics	TEXT	-	是	-	歌词内容, 支持 LRC 格式
features	JSON	-	是	-	MFCC 等音频特 征向量
cover_url	VARCHAR	500	是	-	封面图 URL

表 4.2（续） 音乐表

列名	数据类型	长度	是否为空	默认值	备注
like_count	INT	-	是	0	点赞次数
status	TINYINT	-	是	1	状态标识，1 为正常，0 为下架
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间，更新时自动刷新

(3) 上传记录表用于接收用户上传音频的信息，系统为其

标注所属用户，并在分析结束之后，将提取的音频特征记录其中，便于后续进行推荐处理。上传记录表信息详情如表 4.3 所示。

表 4.3 上传记录表

列名	数据类型	长度	是否为空	默认值	备注
id	BIGINT	-	否	自增	上传记录主键 ID
user_id	BIGINT	-	否	-	所属用户 ID，外键关联 user.id
file_url	VARCHAR	500	否	-	上传文件 URL，存储于 MinIO
file_name	VARCHAR	255	否	-	原始文件名称
file_size	BIGINT	-	是	-	文件大小，单位为字节
file_type	VARCHAR	50	是	-	文件类型
status	VARCHAR	20	是	'UPLOADING'	上传与分析状态
features	JSON	-	是	-	提取的音频特征数据
analysis_result	TEXT	-	是	-	AI 分析结果描述
error_message	VARCHAR	500	是	-	处理失败时的错误信息
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间，更新时自动刷新

(4) 推荐记录表根据上传记录表中获取的音频特征，与音乐表内相似度进行匹配，生成对应的推荐顺序以及匹配度反馈给用户，同

时记录每一个推荐的创建时间，以使用户更好的浏览。推荐记录表详细信息如表 4.4 所示。

表 4.4 推荐记录表

列名	数据类型	长度	是否为空	默认值	备注
----	------	----	------	-----	----

列名	数据类型	长度	是否为空	默认值	备注	(5) 反
id	BIGINT	-	否	自增	推荐记录主键 ID	馈及社
upload_id	BIGINT	-	否	-	所属上传记录 ID, 外键关联 upload_record.id	区表记
music_id	BIGINT	-	否	-	推荐音乐 ID, 外键关联 music.id	录着用
similarity	DECIMAL	5,4	否	-	相似度分数, 取值范围为 0 到 1	户的基
rank	INT	-	否	-	推荐排名	本信息,
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间	将用户
						在社区
						讨论或
						是对系
						统反馈

部分的信息进行记录, 由后台管理员进行查看。反馈及社区表详细信息如表 4.5 所示。

表 4.5 反馈及社区表

列名	数据类型	长度	是否为空	默认值	备注	(6) 评
id	BIGINT	-	否	自增	反馈主键 ID	论点赞
user_id	BIGINT	-	否	-	用户 ID, 外键关联 user.id	表记录
recommend_id	BIGINT	-	是	-	推荐记录 ID, 外键关联 recommend_record.id	了用户
music_id	BIGINT	-	是	-	音乐 ID, 外键关联 music.id	在歌曲
parent_id	BIGINT	-	是	-	父评论 ID, 用于评论回复	中评论
type	VARCHAR	20	否	-	反馈类型, 如 LIKE、DISLIKE、COMMENT、FAVORITE	的内容

表 4.5 (续) 反馈及社区表

列名	数据类型	长度	是否为空	默认值	备注	以及点
content	TEXT	-	是	-	评论或反馈内容	赞记录,
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间	通过处

赞表详细如表 4.6 所示。

表 4.6 评论点赞表

列名	数据类型	长度	是否为空	默认值	备注	(7) A
id	BIGINT	-	否	自增	反馈主键 ID	I 配置表
user_id	BIGINT	-	否	-	用户 ID, 外键关联 user.id	记录 了
recommend_id	BIGINT	-	是	-	推荐记录 ID, 外键关联 recommend_reco rd.id	接入 AI 的密钥、
music_id	BIGINT	-	是	-	音乐 ID, 外键关联 music.id	接口 地
parent_id	BIGINT	-	是	-	父评论 ID, 用于 评论回复	址、名称
type	VARCHAR	20	否	-	反馈类型, 如 LIKE、DISLIKE、COMMENT、FAVORITE	等信息,
content	TEXT	-	是	-	评论或反馈内容	这 些 信
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间	息 由 管

直接监管。AI 配置表详细如表 4.7 所示。

表 4.7 AI 配置表

列名	数据类型	长度	是否为空	默认值	备注
表 4.7 (续) AI 配置表					
列名	数据类型	长度	是否为空	默认值	备注
id	BIGINT	-	否	自增	AI 配置主键 ID
provider	VARCHAR	50	否	-	AI 服务提供商
api_key	VARCHAR	500	否	-	API 密钥, 按加密方式存储
base_url	VARCHAR	500	是	-	API 基础调用地址
model	VARCHAR	100	否	-	模型名称
temperature	DECIMAL	3,2	是	-	温度参数
timeout	INT	-	是	-	超时时间, 单位为毫秒
analysis_strategy	VARCHAR	20	是	'description'	分析策略, 如 description 或 transcribe
is_active	TINYINT	-	是	1	是否激活, 1 为启用, 0 为禁用

表 4.7（续） AI 配置表

列名	数据类型	长度	是否为空	默认值	备注
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间，更新时自动刷新

(8) 报告表从结果记录了推荐内容，

给推荐的信息提供了一个展示模板，并对模板进行了配置，方便展示。报告表详细如表 4.8 所示。

表 4.8 报告表

列名	数据类型	长度	是否为空	默认值	备注
id	BIGINT	-	否	自增	报告模板主键 ID
name	VARCHAR	100	否	-	模板名称
description	TEXT	-	是	-	模板描述
config	JSON	-	是	-	模板配置，包含字段与布局信息
is_default	TINYINT	-	是	0	是否默认模板，1 为是，0 为否
status	TINYINT	-	是	1	状态标识，1 为启用，0 为禁用
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间，更新时自动刷新

(9) 数据表包含了公告数据表，轮播图数据表以及用户标签数据表三个模块，公告数据

表用于保存后台公告信息和发布状态，轮播图数据表用于保存轮播图、横幅和热点内容配置，用户标签数据表用于保存用户自定义偏好标签，详细如表 4.9、表 4.10、表 4.11 所示。

表 4.9 公告数据表

列名	数据类型	长度	是否为空	默认值	备注
id	BIGINT	-	否	自增	公告主键 ID
title	VARCHAR	200	否	-	公告标题
content	TEXT	-	否	-	公告内容
type	VARCHAR	20	是	'SYSTEM'	公告类型，如 SYSTEM、UPDATE、NEW_MUSIC

列名	数据类型	长度	是否为空	默认值	备注
status	TINYINT	-	是	1	发布状态, 1 为已发布, 0 为草稿
publish_time	DATETIME	-	是	-	发布时间
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间, 更新时间自动刷新

表 4.10 轮播图数据表

表 4.11 用户标签数据表

列名	数据类型	长度	是否为空	默认值	备注
id	BIGINT	-	否	自增	站点内容主键
id	BIGINT	-	否	自增	标签主键 ID
user_id	BIGINT	50	否	=	内容类型, 如 carousel、banner、hot_playlists、banner
type	VARCHAR	50	否	-	所属用户 ID, 外键关联 user_id
name	VARCHAR	50	否	-	标签名称
title	VARCHAR	200	是	-	内容标题
color	VARCHAR	20	是	'#409EFF'	标签颜色, 使用十六进制色值
content	TEXT	-	是	-	内容 JSON 数据
sort_order	INT	-	是	0	标签排序顺序
image_url	VARCHAR	500	是	-	图片 URL
link_url	VARCHAR	500	是	-	跳转链接 URL
sort_order	INT	-	是	0	排序值
status	TINYINT	-	是	1	状态标识, 1 为启用, 0 为禁用
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间, 更新时间自动刷新
列名	数据类型	长度	是否为空	默认值	备注

表 4.11（续） 用户标签数据表

列名	数据类型	长度	是否为空	默认值	备注
create_time	DATETIME	-	是	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	是	CURRENT_TIMESTAMP	更新时间，更新时间自动刷新

4.5
本章小结
本

章在系统分析基础上完成了总体架构设计、功能模块设计、界面关系设计和数据库设计，并围绕六个核心用例给出了活动图与时序图的对应关系。由此形成了从业务逻辑到数据结构、从前端页面到后台支撑能力的统一设计方案。

5 系统实现

5.1 系统总体功能说明

本章围绕系统中的六个模块展开实现说明,包括用户认证与资料维护模块、音频 AI 推荐模块、音乐浏览播放与收藏模块、社区互动模块、管理员内容资源管理模块和用户、社区与 AI 管理模块。系统采用前后端分离架构,用户端主要用于音乐浏览、播放、上传分析、收藏与社区互动;管理端主要用于用户维护、曲库管理、社区管理、公告管理、站点运营与 AI 管理。后端通过控制器暴露 REST 风格接口,服务层完成业务逻辑编排,Mapper 层完成数据库访问,Redis 与 MinIO 分别承担状态缓存与对象存储职责,Spring AI 模块则负责音频分析与推荐解释生成。

5.2 功能模块实现

5.2.1 用户认证与资料维护模块

用户认证与资料维护功能主要面向用户注册、登录、找回密码和个人资料维护场景。登录时,系统通过用户名和密码完成校验,若用户状态正常则生成 JWT 令牌并返回给前端;注册时,系统在校验邮箱验证码和用户名唯一性后完成用户创建;找回密码功能则借助邮箱验证码和密码重置流程实现。资料维护模块支持用户查看和修改个人资料、上传头像以及修改密码。

这一模块在实现上有两个特点。其一,认证与资料维护被拆分为公开接口和登录后接口,既保证找回密码等流程对未登录用户开放,又保证资料修改必须建立在已认证基础上。其二,头像资源通过 MinIO 存储,数据库仅保存资源地址,避免将文件二进制内容直接写入数据库。详情如图 5.1 和图 5.2 所示。

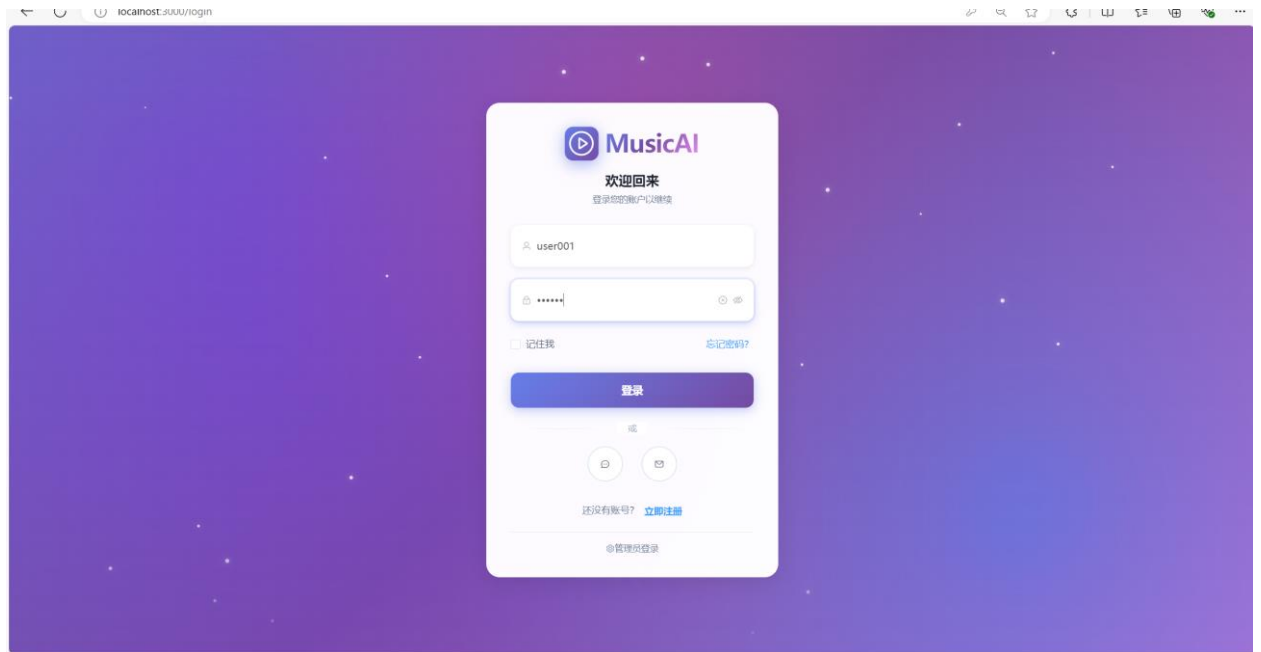


图 5.1 用户认证界面

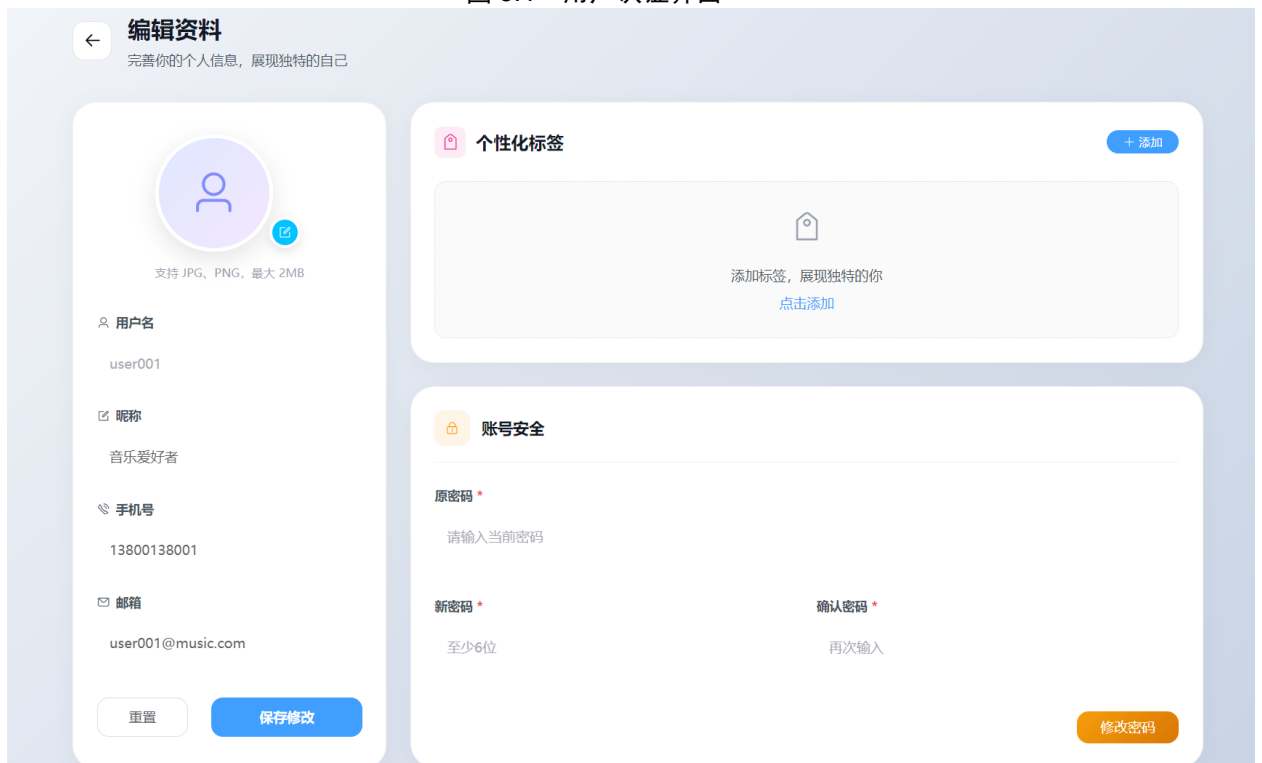


图 5.2 用户资料界面

用户认证与维护部分代码如下：

```
@PostMapping("/login")
```

```
public Result<LoginResponse> login(@RequestBody LoginRequest request) {
```

```
    try {
```

```
        User user = userService.login(request.getUsername(), request.getPassword());
```

```
        if (user == null) {
```

```

        return Result.error("用户名或密码错误");
    }
    if (user.getStatus() == 0) {
        return Result.error("账号已被禁用");
    }
    String token = jwtUtil.generateToken(user.getId(), user.getUsername(),
user.getRole());
    LoginResponse response = new LoginResponse();
    response.setToken(token);
    response.setUserId(user.getId());
    response.setUsername(user.getUsername());
    response.setRole(user.getRole());
    response.setNickname(user.getNickname());
    response.setAvatar(user.getAvatar());

    if (user.getEmail() != null && !user.getEmail().isEmpty()) {
        verificationCodeService.deleteCode(user.getEmail(),
"RESET_PASSWORD");
    }
    return Result.success("登录成功", response);
} catch (Exception e) {
    log.error("登录失败", e);
    return Result.error("登录失败: " + e.getMessage());
}
}

```

5.2.2 音频 AI 推荐模块

音频 AI 推荐功能主要面向用户上传个人音频并获取相似歌曲推荐的场景。用户在上传分析页选择音频文件后，前端首先调用音频上传接口提交文件，后端在上传控制器中接收文件，再由文件服务校验音频格式与大小，将文件保存到 MinIO，并在上传记录表中创建状态为“上传中”的记录。随后前端调用音频分析接口触发分析流程，系统先校验上传记录归属关系，再进入推荐服务完成 AI 分析和推荐生成。

分析过程中，推荐服务会先把上传记录状态更新为“分析中”，然后调用音频特征提取逻辑，根据当前激活的 AI 配置生成结构化音乐描述、多维特征向量和推荐理由，再将这些结果写回上传记录表。系统随后遍历曲库中已有的音乐特征，利用余弦相似度计算相似歌曲，只保留相似度超过阈值的记录并按相似度排序后写入推荐记录表。前端最后通过推荐结果查询接口和上传历史查询接口读取分析状态、推荐理由和推荐列表，在分析结果页和推荐历史页中进行展示。该实现方式把文件上传、AI 分析、相似度计算和结果持久化拆分为清晰阶段，便于失败重试、历史追踪和后台监控。详情如图 5.3 所示。

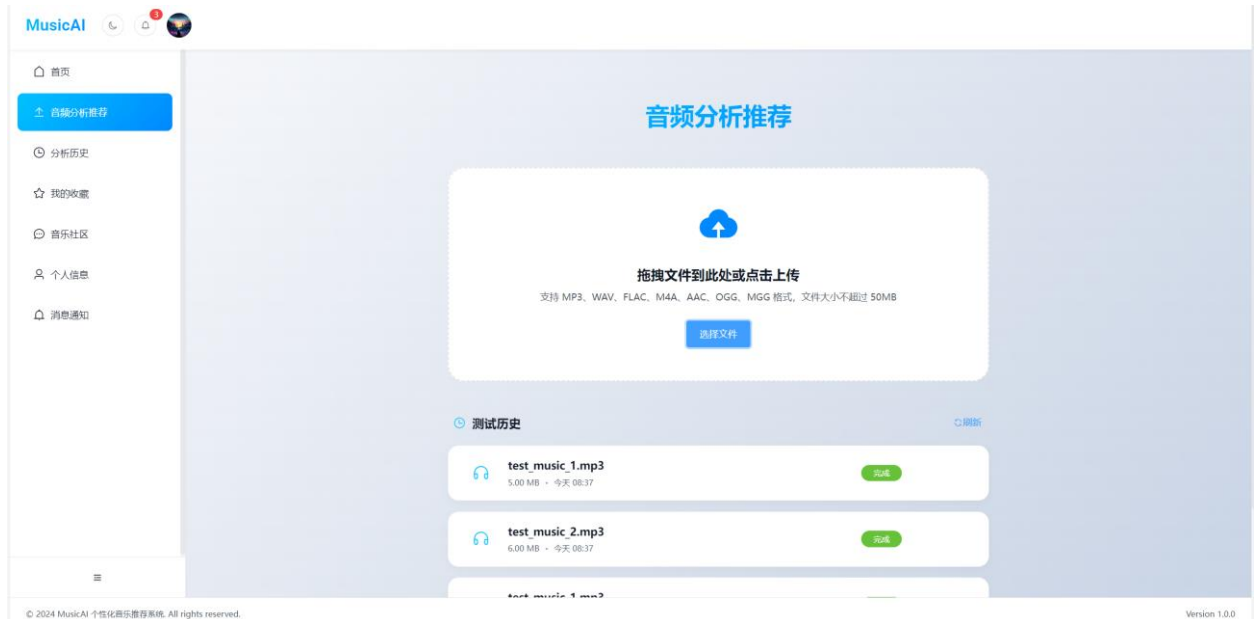


图 5.3 音乐上传界面

音频 AI 推荐模块部分代码如下：

```
@PostMapping("/upload")
```

```
public Result<Map<String, Object>> uploadFile(@RequestParam("file") MultipartFile
file,
```

```
@RequestAttribute Long userId) {
```

```
UploadRecord record = uploadService.saveUploadRecord(userId, file);
```

```
Map<String, Object> result = new HashMap<>();
```

```
result.put("uploadId", record.getId());
```

```
result.put("fileUrl", record.getFileUrl());
```

```
result.put("fileName", record.getFileName());
```

```
result.put("fileSize", record.getFileSize());
```

```
result.put("status", record.getStatus());
```

```
return Result.success("上传成功", result);
```

```
}

@PostMapping("/analyze/{uploadId}")
public Result<String> analyzeAudio(@PathVariable Long uploadId,
                                   @RequestAttribute Long userId) {
    UploadRecord record = uploadService.getById(uploadId);
    if (record == null || !record.getUserId().equals(userId)) {
        return Result.error("上传记录不存在");
    }
    recommendService.analyzeAndRecommend(uploadId);
    return Result.success("分析完成，推荐已生成", null);
}

@Transactional
public void analyzeAndRecommend(Long uploadId) {
    UploadRecord uploadRecord = uploadRecordMapper.selectById(uploadId);
    if (uploadRecord == null) {
        throw new RuntimeException("上传记录不存在");
    }
    try {
        uploadRecord.setStatus("ANALYZING");
        uploadRecordMapper.update(uploadRecord);
        String features =
audioAnalysisService.extractFeatures(uploadRecord.getFileUrl());
        String recommendationReason = generateRecommendationReason(features);

        uploadRecord.setFeatures(features);
        uploadRecord.setAnalysisResult(recommendationReason);
        uploadRecord.setStatus("COMPLETED");
        uploadRecordMapper.update(uploadRecord);
        generateRecommendations(uploadId, features);
    } catch (Exception e) {
        uploadRecord.setStatus("FAILED");
    }
}
```



```

        uploadRecord.setErrorMessage(e.getMessage());
        uploadRecordMapper.update(uploadRecord);
        throw e;
    }
}

```

5.2.3 音乐浏览播放与收藏模块

音乐浏览播放与收藏功能主要面向用户在首页、搜索页、歌曲详情页和收藏页中的持续听歌场景。用户在首页输入关键字、歌手或风格后，前端调用音乐列表查询接口获取分页曲库数据；点击歌曲卡片后，详情页再调用歌曲详情查询接口、歌曲评论查询接口和试听地址查询接口拉取歌曲信息、评论和试听地址，供页面展示封面、歌词、评论区和播放器组件使用。后端在音乐控制器与音乐服务中完成曲库查询，并只返回状态正常的音乐资源给前端。详情如图 5.4 所示。

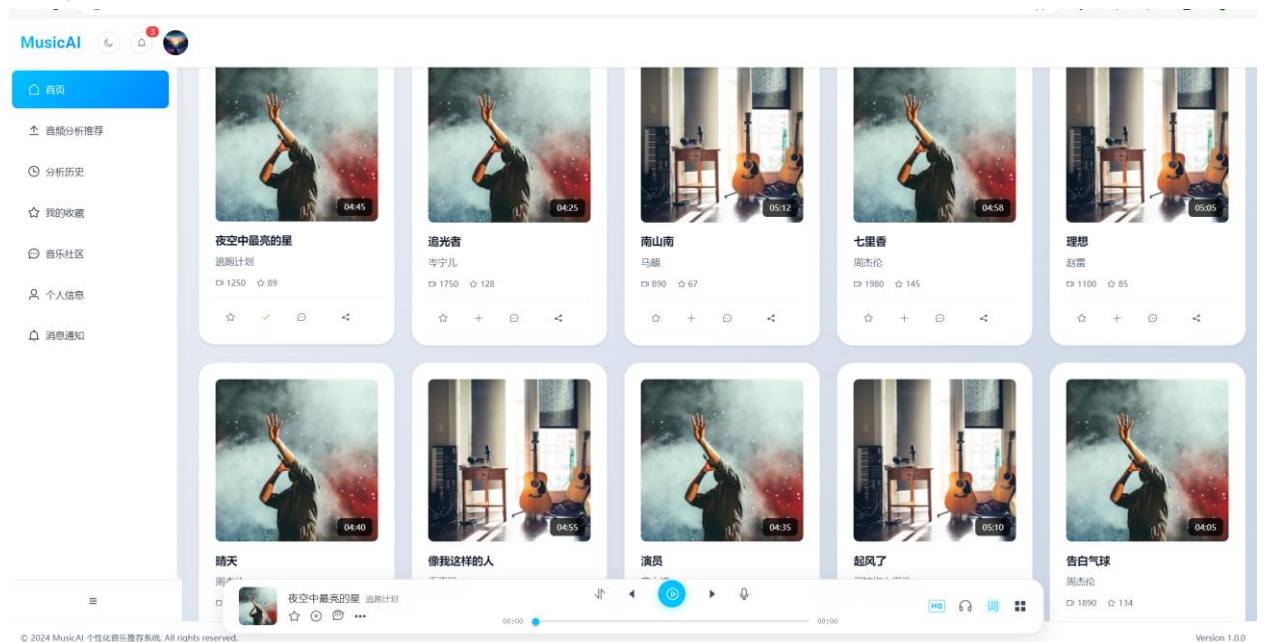


图 5.4 音乐浏览界面

音乐浏览播放与收藏模块部分代码如下：

```

@GetMapping("/music/list")
public Result<Map<String, Object>> getMusicList(
    @RequestParam(required = false) String keyword,
    @RequestParam(required = false) String artist,
    @RequestParam(required = false) String genre,
    @RequestParam(defaultValue = "10") Integer pageSize) {

```

```
        Map<String, Object> result = musicService.getMusicList(
            keyword, artist, genre, 1, pageNum, pageSize
        );
        return Result.success(result);
    }

    @PostMapping("/music/{musicId}/favorite")
    public Result<Map<String, Object>> toggleFavorite(@PathVariable Long musicId,
                                                       @RequestAttribute
Long userId) {
        boolean isFavorite = feedbackService.toggleFavorite(musicId, userId);
        Map<String, Object> result = new HashMap<>();
        result.put("isFavorite", isFavorite);
        return Result.success(isFavorite ? "收藏成功" : "已取消收藏", result);
    }

    @PostMapping("/playlist/state/save")
    public Result<String> savePlayState(@RequestAttribute Long userId,
                                        @RequestBody PlayStateRequest request)
    {
        playlistService.savePlayState(
            userId,
            request.getCurrentMusicId(),
            request.getCurrentIndex(),
            request.getPlayMode(),
            request.getCurrentTime(),
            request.getIsPlaying()
        );
        return Result.success("保存播放状态成功");
    }
}
```

5.2.4 社区互动模块

社区互动与反馈功能主要面向歌曲评论区、社区动态页和推荐结果评价场景。用户在歌曲详情页输入评论内容后，前端调用评论提交接口写入评论；若用户在评论区中回复其他用户，则调用评论回复接口，后端把回复内容仍写入反馈表，并通过父评论编号构建父子评论关系。社区页加载时，前端调用社区动态查询接口获取聚合后的动态流，系统按照最新或最热规则查询顶层评论，并补充歌曲标题、歌手、封面、点赞状态等信息后返回页面展示。详情如图 5.5 所示。

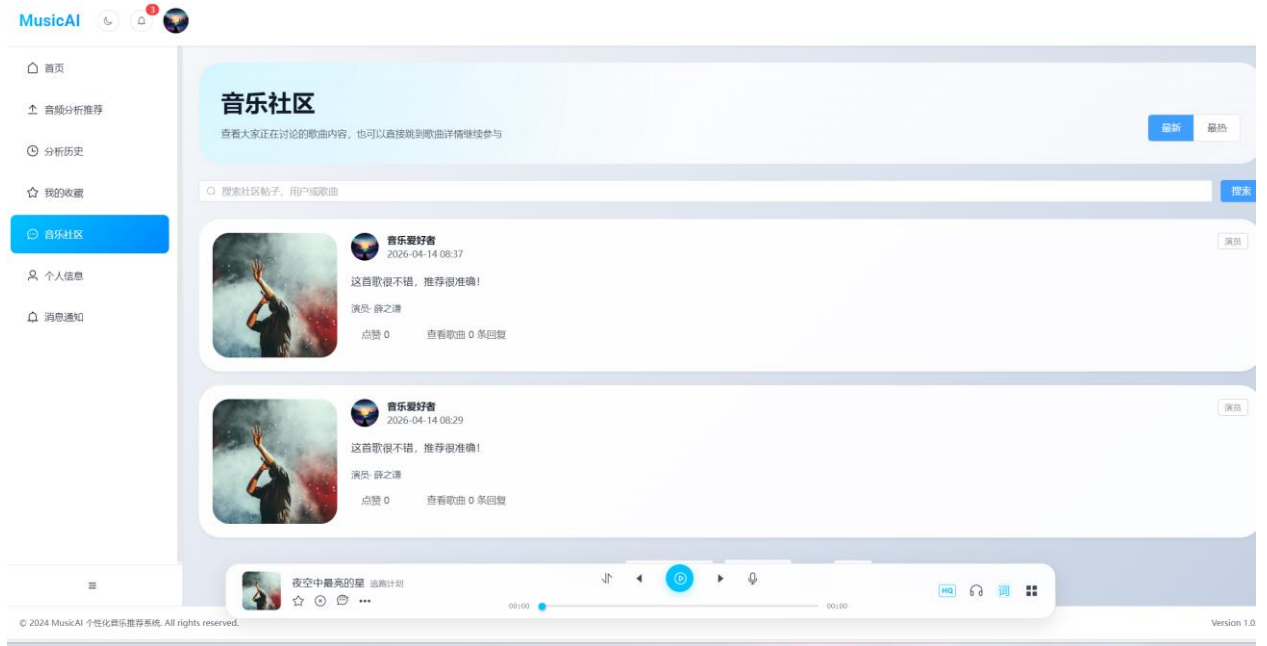


图 5.5 社区界面

社区互动模块部分代码如下：

```
@GetMapping("/community")
public Result<Map<String, Object>> getCommunityFeed(@RequestAttribute Long
userId,
@RequestParam(required = false) String keyword,
@RequestParam(defaultValue = "1") Integer pageNum,
@RequestParam(defaultValue = "10") Integer pageSize,
@RequestParam(defaultValue = "latest") String sortType) {
    List<CommunityPostVO> list = feedbackService.getCommunityFeed(
        userId, keyword, pageNum, pageSize, sortType);
    long total = feedbackService.countCommunityFeed(keyword);

    Map<String, Object> result = new HashMap<>();
```

```
result.put("list", list);
result.put("total", total);
result.put("pageNum", pageNum);
result.put("pageSize", pageSize);
result.put("sortType", sortType);
return Result.success(result);
}
```

```
@PostMapping("/music/{musicId}/comment")
public Result<String> submitComment(@PathVariable Long musicId,
                                     @RequestBody Map<String, String> request,
                                     @RequestAttribute Long userId) {
    String content = request.get("content");
    if (content == null || content.trim().isEmpty()) {
        return Result.error("评论内容不能为空");
    }
}
```

```
Feedback feedback = new Feedback();
feedback.setUserId(userId);
feedback.setMusicId(musicId);
feedback.setType("COMMENT");
feedback.setContent(content.trim());
boolean success = feedbackService.submitFeedback(feedback);

return success ? Result.success("评论成功", null) : Result.error("评论失败");
}
```

```
@PostMapping("/comment/{commentId}/like")
public Result<Map<String, Object>> toggleCommentLike(@PathVariable Long
commentId,
                                                       @RequestAttribute Long userId) {
    boolean success = feedbackService.toggleCommentLike(commentId, userId);
}
```

```
boolean isLiked = feedbackService.checkCommentLike(commentId, userId);  
Map<String, Object> result = new HashMap<>();  
result.put("isLiked", isLiked);  
result.put("success", success);  
return Result.success(isLiked ? "点赞成功" : "已取消点赞", result);  
}
```

5.2.5 内容资源管理模块

内容资源管理功能主要面向管理员侧的曲库维护、首页运营配置和公告发布场景。管理员在音乐管理页填写歌曲名称、歌手、专辑、风格和歌词信息，并上传音频文件和封面图片后，前端以表单加文件的方式调用音乐新增接口；后端先校验文件格式与大小，再通过文件服务将音频和封面上传到 MinIO，随后把资源地址和歌曲元数据写入音乐表。站点内容管理页则通过站点内容维护接口管理轮播图、横幅和热点内容，公告管理页通过公告维护接口管理公告标题、内容、类型、状态和发布时间。详情如图 5.6 和图 5.7 所示。

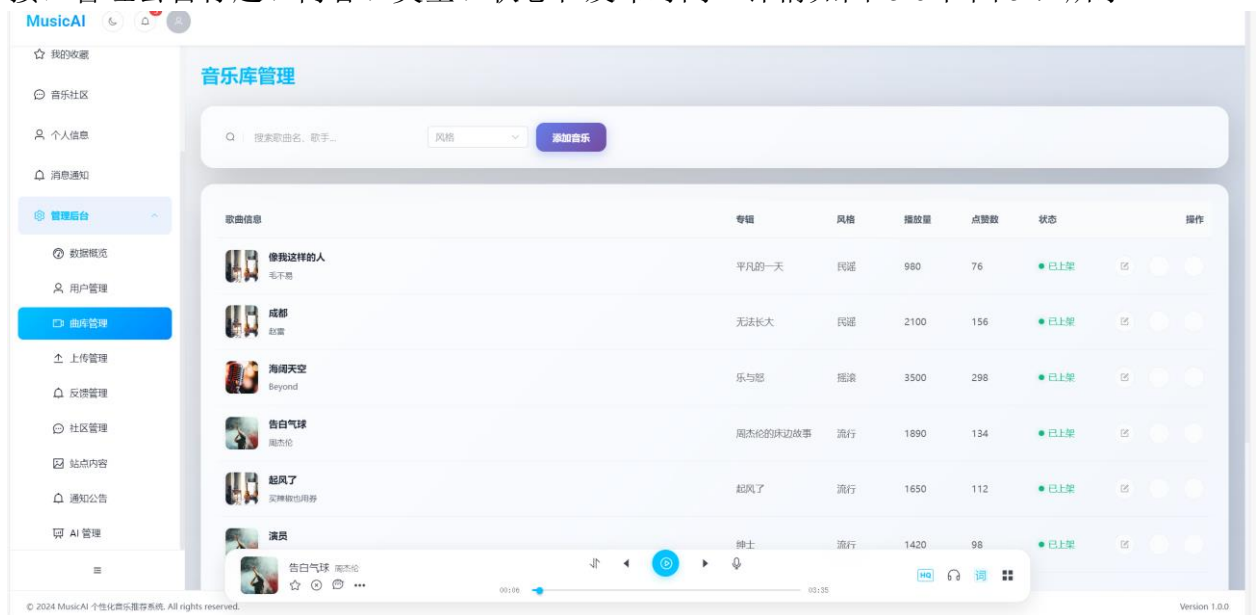


图 5.6 曲库界面

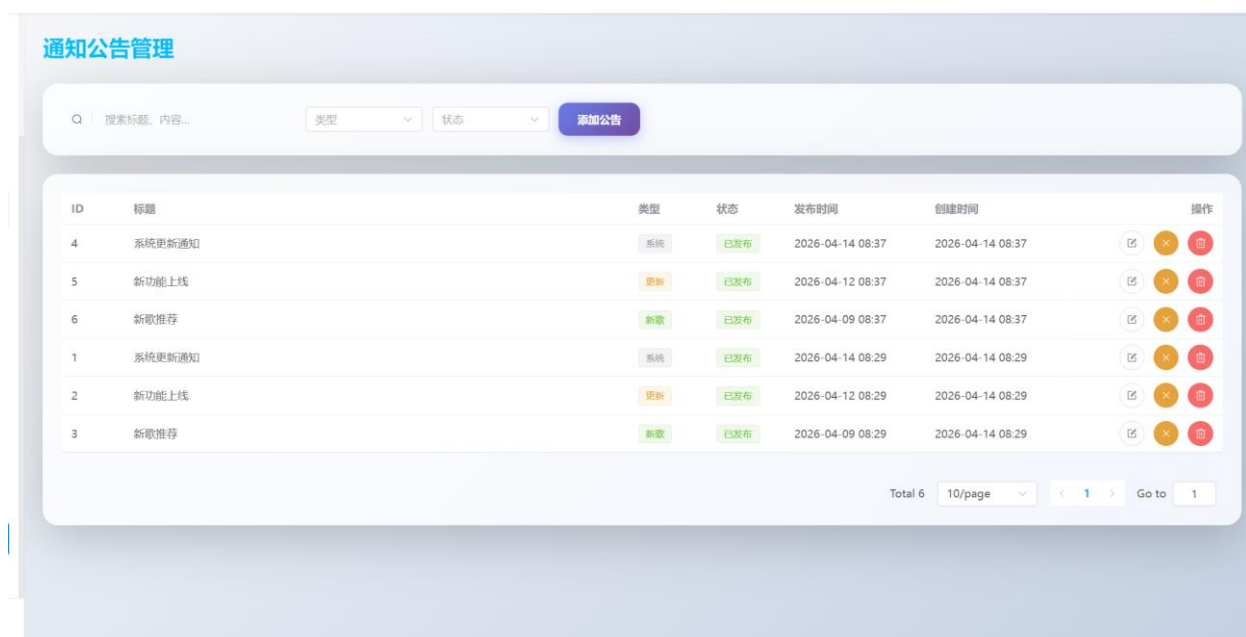


图 5.7 公告界面

内容资源管理部分代码如下：

```
@PostMapping("/api/admin/music")
public Result<Map<String, Object>> addMusic(@RequestParam("title") String title,
    @RequestParam("artist") String artist,
    @RequestParam("audioFile") MultipartFile audioFile,
    @RequestParam(value = "album", required = false) String album,
    @RequestParam(value = "genre", required = false) String genre,
    @RequestParam(value = "lyrics", required = false) String lyrics,
    @RequestParam(value = "coverFile", required = false) MultipartFile coverFile,
    @RequestParam(value = "coverUrl", required = false) String coverUrl) {
    if (title == null || title.trim().isEmpty()) {
        return Result.error("歌曲名不能为空");
    }
    if (artist == null || artist.trim().isEmpty()) {
        return Result.error("歌手不能为空");
    }
    if (audioFile == null || audioFile.isEmpty()) {
        return Result.error("音频文件不能为空");
    }
}
```

```
Map<String, String> audioInfo = fileService.saveLibraryFile(audioFile, null);
String coverFileUrl = null;
if (coverFile != null && !coverFile.isEmpty()) {
    coverFileUrl = fileService.saveCoverImage(coverFile, null).get("fileUrl");
} else if (coverUrl != null && !coverUrl.isEmpty()) {
    coverFileUrl = coverUrl.trim();
}
```

```
Music music = new Music();
music.setTitle(title.trim());
music.setArtist(artist.trim());
music.setAlbum(album != null ? album.trim() : null);
music.setGenre(genre);
music.setLyrics(lyrics != null && !lyrics.trim().isEmpty() ? lyrics.trim() : null);
music.setFileUrl(audioInfo.get("fileUrl"));
music.setCoverUrl(coverFileUrl);
music.setStatus(1);
```

```
boolean success = musicService.addMusic(music);
if (!success) {
    fileService.deleteFile(audioInfo.get("fileUrl"));
    if (coverFileUrl != null) {
        fileService.deleteFile(coverFileUrl);
    }
    return Result.error("添加音乐信息失败");
}
```

```
Map<String, Object> result = new HashMap<>();
result.put("musicId", music.getId());
result.put("fileUrl", music.getFileUrl());
result.put("coverUrl", music.getCoverUrl());
```

```
        return Result.success("歌曲添加成功", result);
    }

    @PostMapping("/api/admin/site-content")
    public Result<Map<String, Object>> addContent(@RequestParam String type,
    @RequestParam(required = false) String title,
    @RequestParam(required = false) MultipartFile imageFile,
    @RequestParam(required = false) String imageUrl,
    @RequestParam(required = false) String linkUrl,
    @RequestParam(required = false) String content,
    @RequestParam(defaultValue = "0") Integer sortOrder) {
        SiteContent siteContent = new SiteContent();
        siteContent.setType(type);
        siteContent.setTitle(title);
        siteContent.setContent(content);
        siteContent.setLinkUrl(linkUrl);
        siteContent.setSortOrder(sortOrder);
        siteContent.setStatus(1);

        if (imageFile != null && !imageFile.isEmpty()) {
            Map<String, String> imageInfo = fileService.saveCoverImage(imageFile,
null);

            siteContent.setImageUrl(imageInfo.get("fileUrl"));
        } else if (imageUrl != null && !imageUrl.isEmpty()) {
            siteContent.setImageUrl(imageUrl);
        }

        siteContentMapper.insert(siteContent);
        Map<String, Object> data = new HashMap<>();
        data.put("contentId", siteContent.getId());
        return Result.success("添加成功", data);
    }
}
```



```
@PostMapping("/api/admin/notices")
public Result<Map<String, Object>> addNotice(@RequestBody Notice notice) {
    if (notice.getStatus() == null) {
        notice.setStatus(0);
    }
    if (notice.getType() == null || notice.getType().isEmpty()) {
        notice.setType("SYSTEM");
    }
    if (notice.getStatus() == 1 && notice.getPublishTime() == null) {
        notice.setPublishTime(LocalDateTime.now());
    }

    boolean success = noticeMapper.insert(notice) > 0;
    if (!success) {
        return Result.error("添加失败");
    }

    Map<String, Object> result = new HashMap<>();
    result.put("noticeId", notice.getId());
    return Result.success("添加成功", result);
}
```

5.2.6 用户、社区与 AI 管理模块

用户、社区与 AI 管理功能主要面向管理员侧的平台治理场景。管理员在用户管理页输入关键字、角色或状态条件后，前端调用用户列表查询接口获取分页用户列表，并可继续调用用户状态更新接口完成禁用或启用操作；在上传管理和反馈管理页中，前端分别调用上传记录查询接口和反馈记录查询接口读取上传记录、评论与反馈数据，并按需执行删除操作，从而完成对异常内容和无效数据的治理。详情如图 5.8 和图 5.9 所示。



图 5.8 用户管理界面

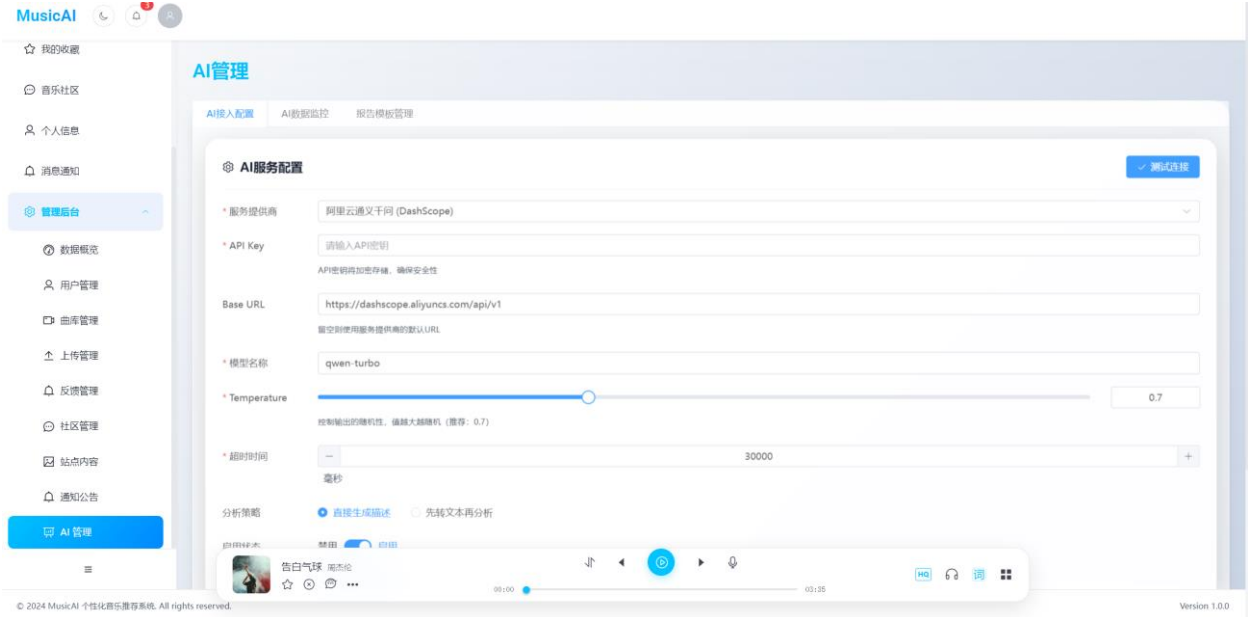


图 5.9 AI 界面

用户、社区与 AI 管理部分代码如下：

```
@GetMapping("/api/admin/users")
public Result<Map<String, Object>> getUserList(@RequestParam(required = false)
String keyword,
@RequestParam(required = false) String role,
@RequestParam(required = false) Integer status,
@RequestParam(defaultValue = "1") Integer pageNum,
@RequestParam(defaultValue = "10") Integer pageSize) {
    int offset = (pageNum - 1) * pageSize;
```

```

        List<User> list = userMapper.selectList(keyword, role, status, offset, pageSize);
        long total = userMapper.count(keyword, role, status);
        list.forEach(user -> user.setPassword(null));

        Map<String, Object> result = new HashMap<>();
        result.put("list", list);
        result.put("total", total);
        result.put("pageNum", pageNum);
        result.put("pageSize", pageSize);
        return Result.success(result);
    }

    @PutMapping("/api/admin/users/{id}/status")
    public Result<String> updateUserStatus(@PathVariable Long id,
                                           @RequestBody Map<String, Integer>
request) {
        Integer status = request.get("status");
        User user = new User();
        user.setId(id);
        user.setStatus(status);

        boolean success = userService.update(user);
        return success ? Result.success("状态更新成功", null) : Result.error("状态更新
失败");
    }

    @PostMapping("/api/admin/ai/config/save")
    public Result<Void> saveConfig(@RequestBody AIConfig config) {
        if (config.getId() != null) {
            if (Boolean.TRUE.equals(config.getIsActive())) {
                aiConfigMapper.deactivateOthers(config.getId());
            }
        }
    }

```

```

        aiConfigMapper.update(config);
    } else {
        if (config.getIsActive() == null) {
            config.setIsActive(false);
        }
        aiConfigMapper.insert(config);
        if (Boolean.TRUE.equals(config.getIsActive())) {
            aiConfigMapper.deactivateOthers(config.getId());
        }
    }
    return Result.success(null);
}

```

```

@GetMapping("/api/admin/ai/accuracy")
public Result<Map<String, Object>> getAccuracyStats() {
    Double avgSimilarityValue =
recommendRecordMapper.selectAverageSimilarity();

    double avgSimilarity = avgSimilarityValue == null ? 0.0 : avgSimilarityValue;
    long totalRecommends = recommendRecordMapper.countAll();
    long positiveFeedback = feedbackMapper.count(null, null, "LIKE");
    long negativeFeedback = feedbackMapper.count(null, null, "DISLIKE");
    long evaluatedFeedback = positiveFeedback + negativeFeedback;

    double accuracy = evaluatedFeedback > 0
        ? positiveFeedback * 100.0 / evaluatedFeedback
        : avgSimilarity * 100.0;

    Map<String, Object> stats = new HashMap<>();
    stats.put("accuracy", Math.min(Math.max(accuracy, 0.0), 100.0));
    stats.put("avgSimilarity", avgSimilarity);
    stats.put("totalRecommends", totalRecommends);
    stats.put("positiveFeedback", positiveFeedback);
}

```

```
stats.put("negativeFeedback", negativeFeedback);  
return Result.success(stats);  
}
```

5.3 本章小结

本章结合系统实际代码，对用户认证、上传推荐、音乐播放收藏、社区互动、内容运营以及后台治理六个核心模块的实现流程进行了展开说明。各模块均按照“前端页面触发请求、后端接口接收请求、服务层完成业务处理、数据库或缓存返回结果”的链路组织，并在文件存储、验证码缓存、播放状态缓存和 AI 分析等关键环节引入了 MinIO、Redis 和 Spring AI 等支撑能力。

从实现效果看，系统不仅完成了用户端和管理端的基础业务闭环，也通过上传记录持久化、反馈体系统一建模、内容资源双写入回滚和 AI 监控聚合等实现细节，增强了系统的可追踪性、可维护性和可运营性。这些实现内容也为下一章的系统测试提供了直接依据。

6 系统测试

6.1 测试方法

本章测试的主要目的是验证系统核心功能在典型业务场景下是否能够稳定、正确地运行，并检查系统输出结果是否符合预期。结合本课题的实现特点，测试重点放在用户登录、上传推荐、浏览播放、社区互动、内容资源管理、AI 管理等核心业务流程上。

本章主要采用黑盒测试方法开展功能验证。测试过程中主要根据第三章提出的六个核心用例设计输入、观察系统返回结果，判断系统是否满足需求分析阶段提出的功能要求。

6.2 功能测试

功能测试对系统的主要功能进行测试，包括用户登录、上传推荐、浏览播放、社区互动、内容资源管理、AI 管理等。测试采用黑盒测试方法，按照测试用例输入测试数据，验证系统输出是否符合预期。

6.3 测试用例设计

6.3.1 提交代码判题测试

(1) 用户提交代码判题的测试用例，详情如下表 6.1 所示。

表 6.1 用户提交代码判题测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE001	用户登录	输入正确用户名与密码	返回 JWT 和用户信息，进入系统首页	输出登录成功，跳转首页	符合
TE002	修改个人资料	登录后修改昵称并上传头像	返回“更新成功”或“头像上传成功”	页面提示成功，资料与头像正确更新	符合

(2) 测试过程

输入正确个人信息，点击登录按钮，进入个人信息页，对头像标签等数据进行修改，等待成功提示与数据更新。详情如图 6.1 和 6.2 所示。



图 6.1 登录成功

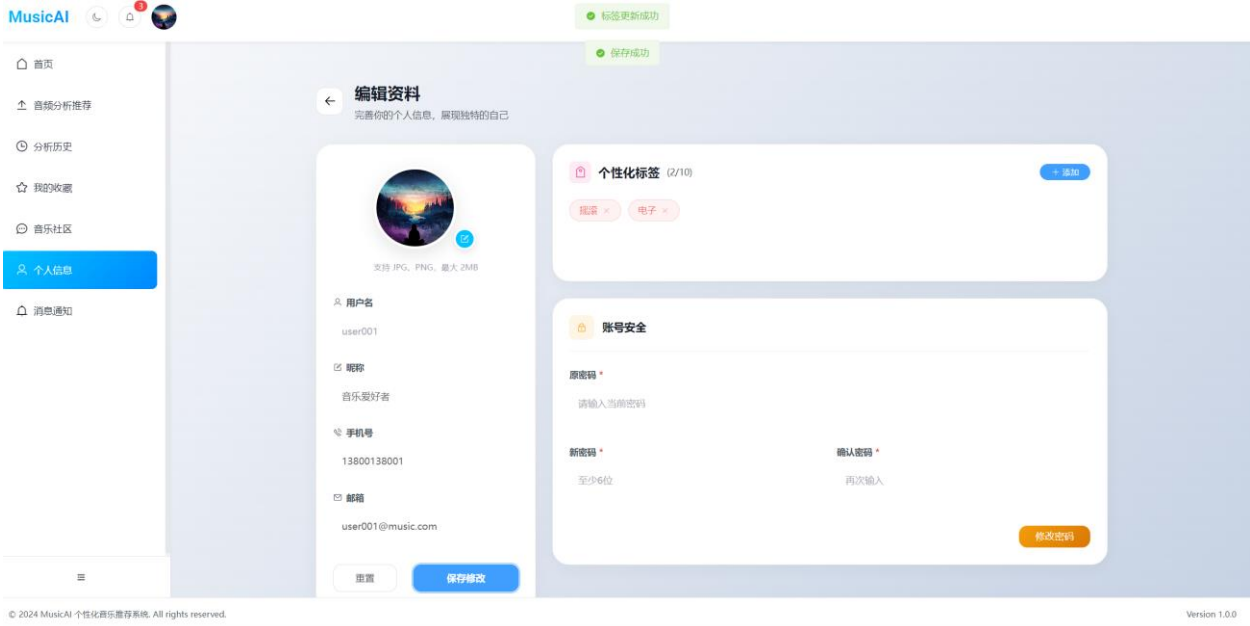


图 6.2 资料修改成功

6.3.2 上传音频与分析测试

(1) 上传音频与分析的测试用例，详情如下表 6.2 所示。

表 6.2 上传音频与分析测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE003	上传合法音频文件	上传 mp3 文件并触发分析	生成上传记录并进入分析流程	上传成功, 记录状态可查询	符合

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE002	查看推荐结果	对已上传音频执行分析	返回推荐理由和推荐列表	页面显示分析描述、相似度和推荐歌曲	符合

(2) 测试过程

上传音频资料，等待分析完成，点击分析记录进入推荐和分析界面。详情如图 6.3、图 6.4 和图 6.5 所示。



图 6.3 音频上传成功



图 6.4 音频分析成功

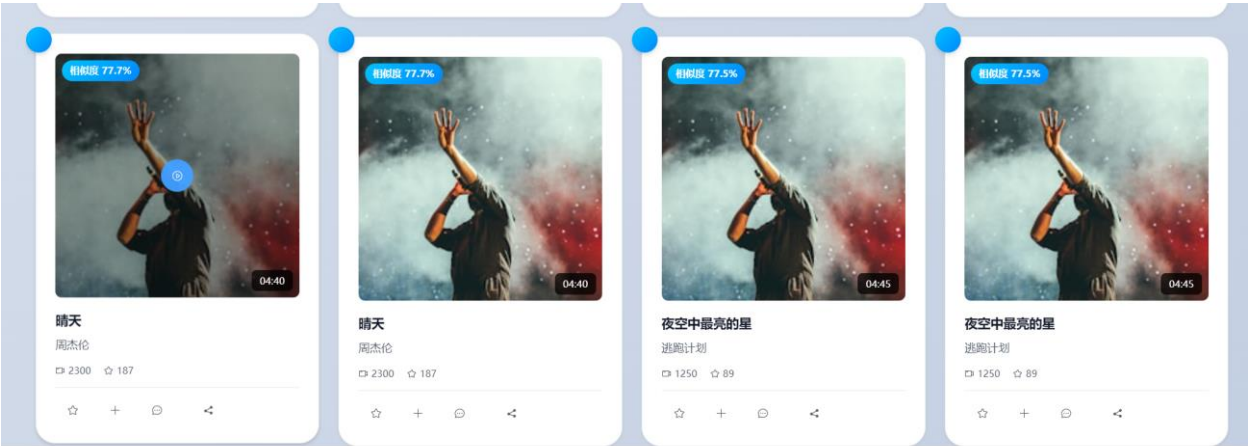


图 6.5 推荐歌曲

6.3.3 音乐浏览播放与收藏测试

(1) 音乐浏览播放与收藏的测试用例，详情如下表 6.3 所示。

表 6.3 音乐浏览播放与收藏测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE005	浏览并播放音乐	在首页选择歌曲进入详情并点击播放	音乐正常播放，播放器状态同步更新	歌曲可播放，播放状态成功保存	符合
TE006	收藏与取消收藏	在歌曲卡片或详情页点击收藏按钮	收藏状态切换，收藏夹内容同步变化	收藏夹列表与按钮状态均正确更新	符合

(2) 测试过程

点击音乐。等待音乐播放，点击收藏按钮，前往我的收藏界面进行查看是否添加。详情如图 6.6 和图 6.7 所示。

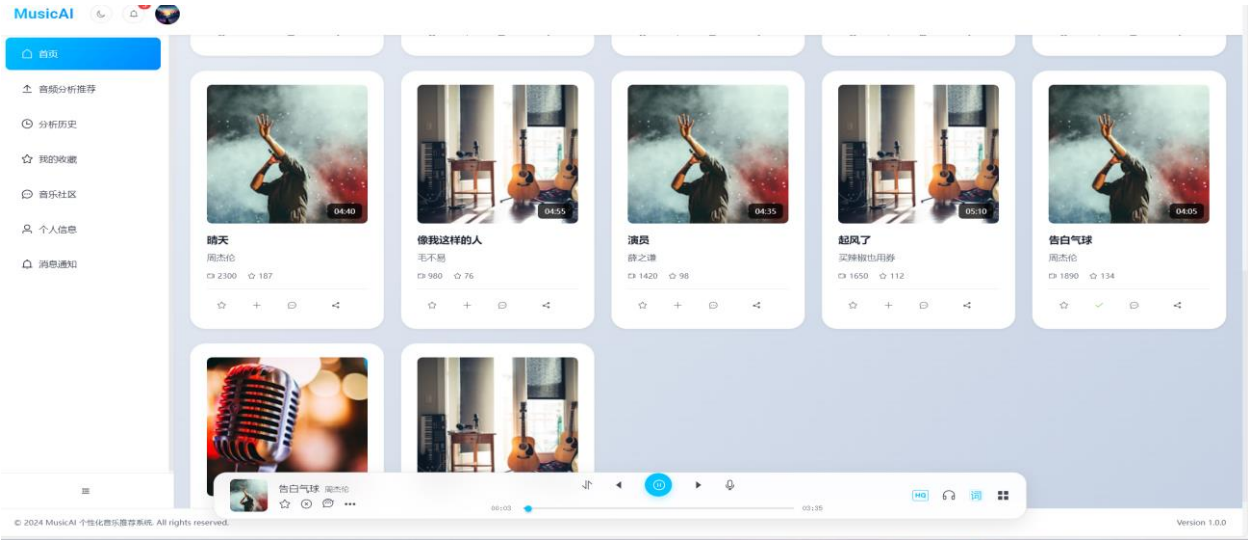


图 6.6 播放测试

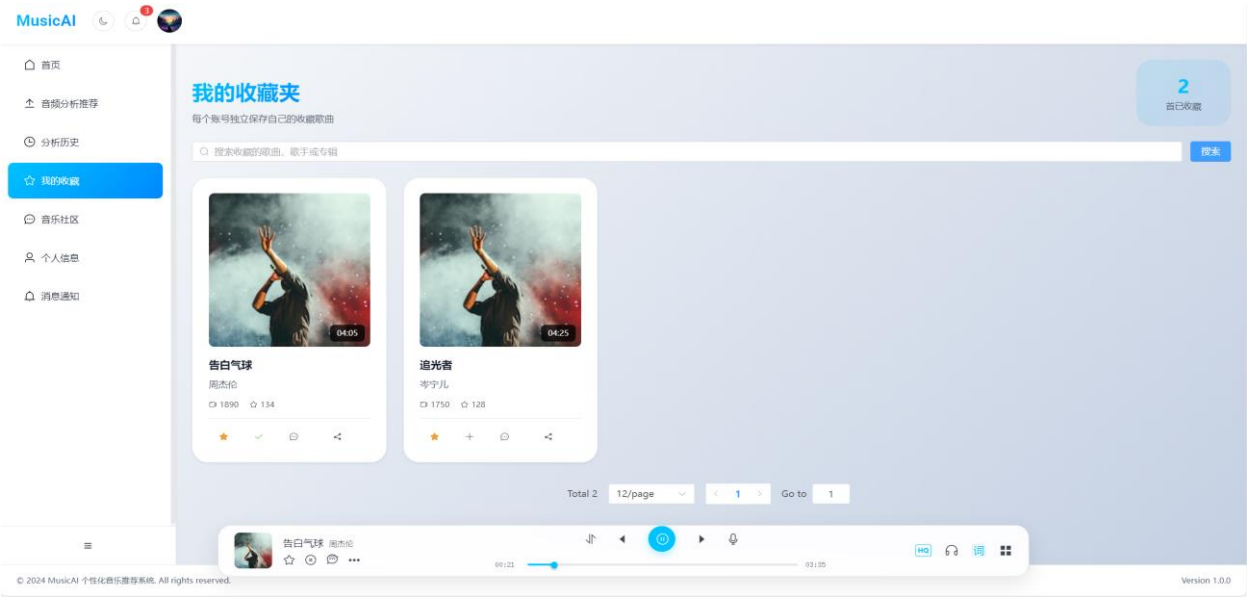


图 6.7 收藏测试

6.3.4 社区互动测试

(1) 社区互动的测试用例，详情如下表 6.4 所示。

表 6.4 社区互动测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE007	发布歌曲评论	在歌曲详情页输入评论内容并提交	评论成功，评论区数量更新	评论列表即时刷新	符合
TE008	社区点赞互动	在社区页对评论执行点赞操作	点赞状态变化且计数更新	社区动态点赞成功并同步显示	符合

(2) 测试过程

进入歌曲评论区，对歌曲发布自己的观点，等待评论显示。对其他用户的评论进行点赞互动，观察是否有记录。详情如图 6.8、图 6.9 和图 6.10 所示。

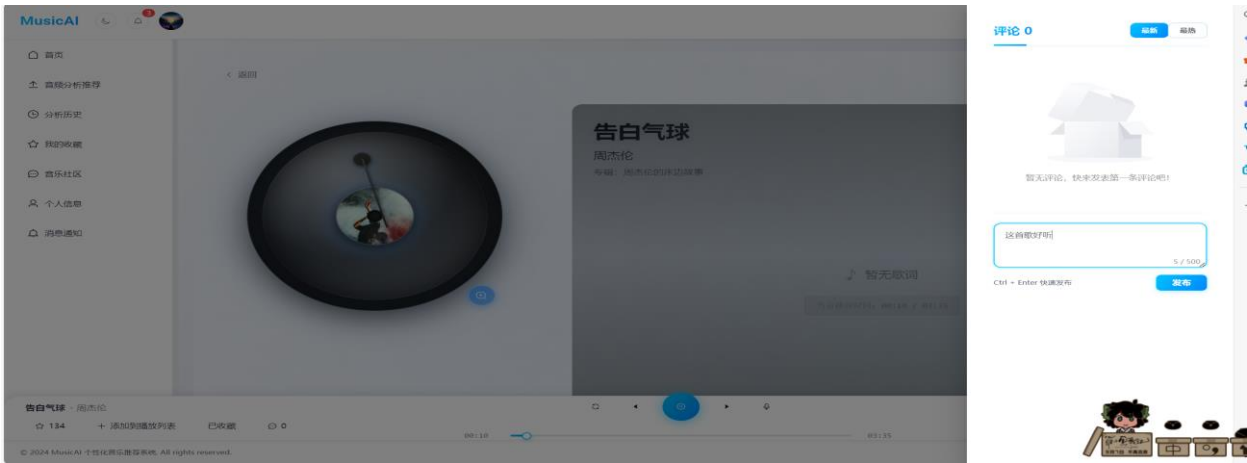


图 6.8 评论

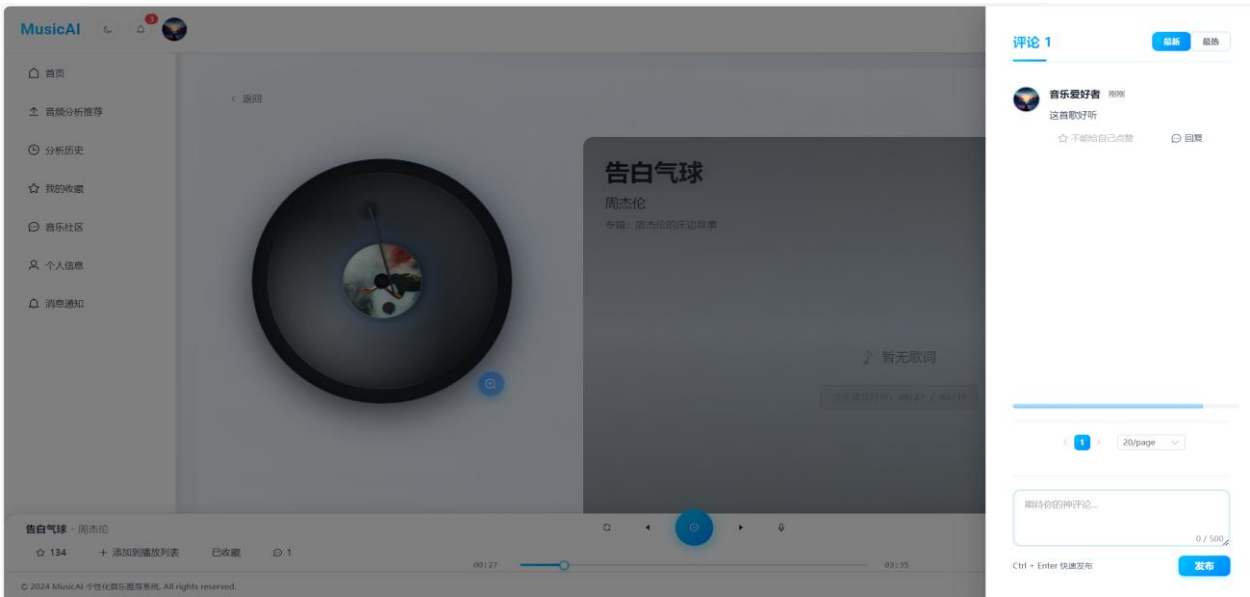


图 6.9 评论成功

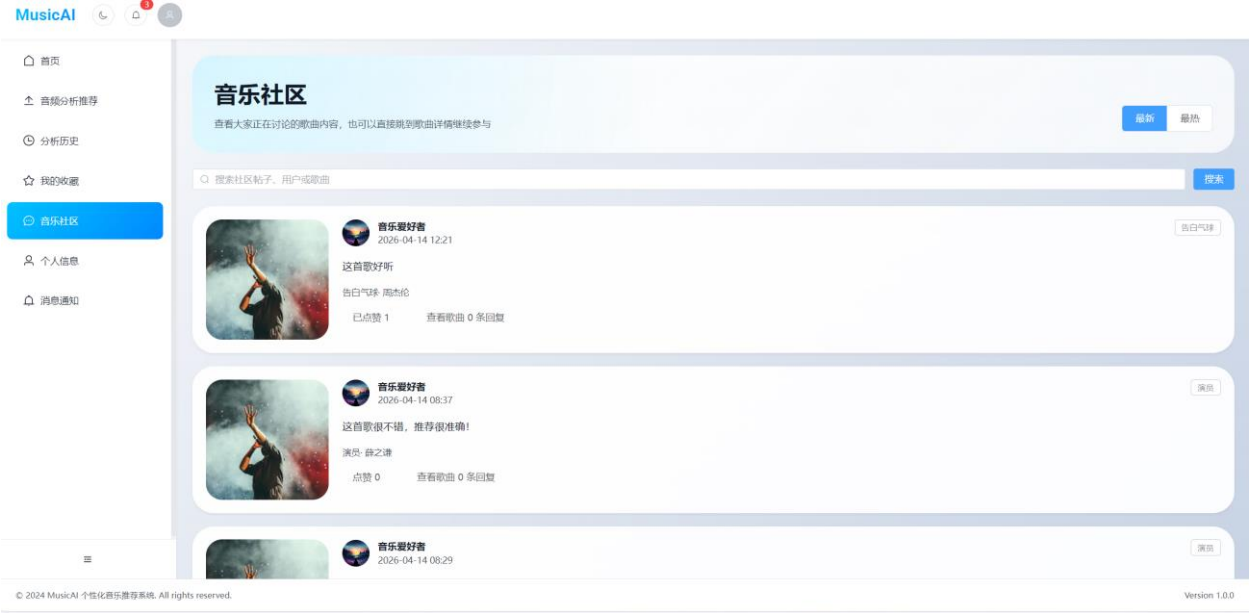


图 6.10 点赞互动成功

6.3.5 内容资源管理测试

(1) 内容资源管理的测试用例，详情如下表 6.5 所示。

表 6.5 内容资源管理测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE009	新增音乐资源	管理员上传音频与封面并填写歌曲信息	音乐记录写入数据库，后台列表可见	新增成功，曲库列表显示正常	符合

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE010	发布公告或轮播内容	管理员新增公告或首页内容	发布后前台可读取对应内容	内容状态更新成功，后台列表可见	符合

(2) 测试过程

管理员上传音频资料，并填写音频名称，作者，以 LRC 歌词形式上传歌词库，对通知内容进行修改，等待同步数据成功。详情如图 6.11 和 6.12 所示。

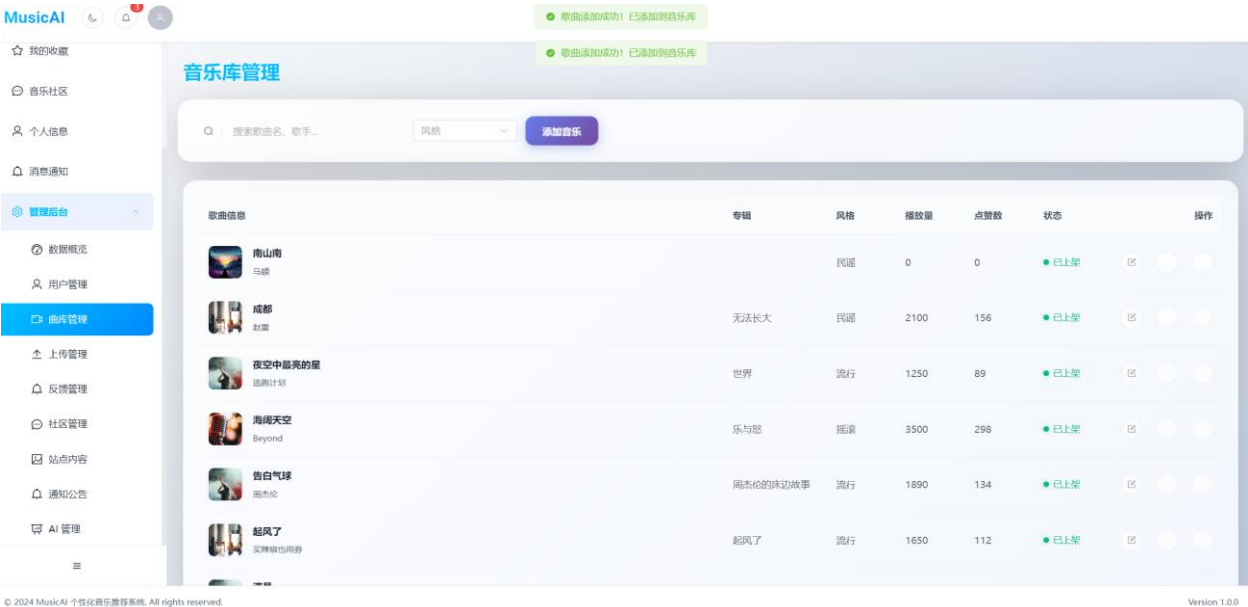


图 6.11 添加歌曲成功

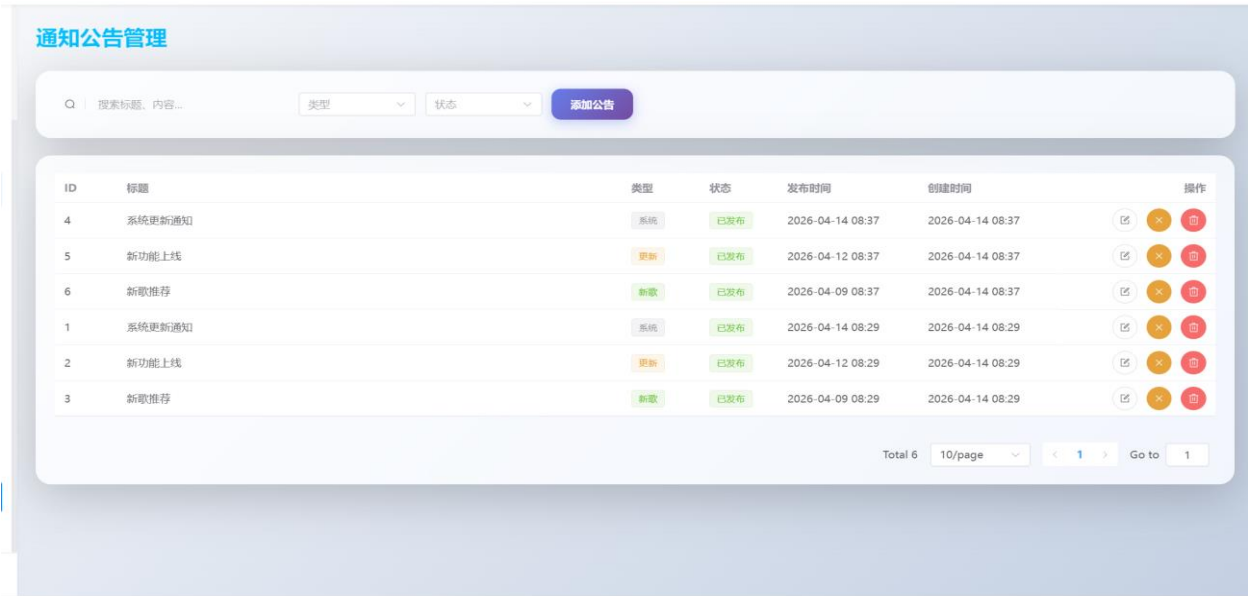


图 6.12 公告添加

6.3.6 用户、社区与 AI 管理测试

(1) 用户、社区与 AI 管理的测试用例，详情如下表 6.6 所示。

表 6.6 用户提交代码判题测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE011	禁用普通用户	在后台用户管理页修改用户状态为禁用	返回状态更新成功，目标用户无法正常登录	提示成功，禁用逻辑生效	符合

表 6.6（续） 用户提交代码判题测试

测试编号	测试项	输入	预期输出	实际输出	测试结论
TE012	AI 配置与监控查询	查看当前 AI 配置并刷新监控数据	返回模型配置、服务状态和统计图数据	页面正常展示配置和监控统计	符合

（2）测试过程

管理员进入用户管理界面，对用户进行禁用操作，数据面板更新，管理员进入 ai 配置界面，配置对应的 api key，显示同步完成。详情如图 6.13 和图 6.14 所示。



图 6.13 禁用用户

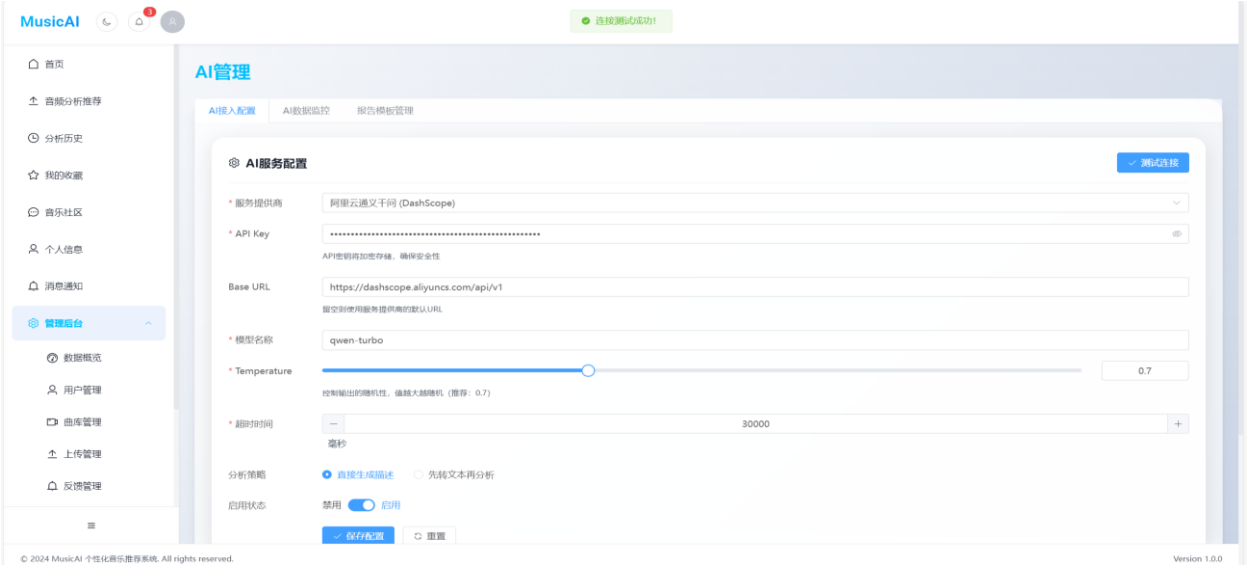


图 6.14 AI 配置成功

6.4 本章小结

本章围绕系统的 6 个核心业务模块开展了测试工作，采用黑盒测试方法设计了多个典型场景的测试用例，对应覆盖了第 3 章用例建模中给出的核心用例。测试结果表明，各功能模块均能得到符合预期的反馈，说明系统在功能层面基本满足需求分析阶段提出的业务要求。

本章测试主要聚焦于功能正确性验证，尚未开展系统性能、压力与安全渗透等方面的专项测试。作为系统主要功能的目标已实现，确保用户能有良好的使用体验。

7 分析、总结与展望

7.1 安全与隐私分析

本系统在安全层面采用了多种手段进行保障。认证层使用 JWT 令牌和角色权限控制，防止未授权访问；找回密码环节引入邮箱验证码机制，增强账号安全；用户密码采用 BCrypt 加密存储，避免明文泄露；文件管理层通过对象存储隔离音频、封面与头像资源，降低数据库直接存储文件带来的风险；社区模块通过后台反馈管理和社区管理功能支持内容治理；AI 模块的配置和启用状态由管理员统一控制，减少错误配置或随意切换造成的系统风险。

7.2 技术经济与成本分析

系统主要依托开源技术栈实现，包括 Spring Boot、Vue 3、MyBatis、MySQL、Redis、MinIO 和 Element Plus 等，因此开发成本和部署成本相对较低。AI 模块虽然引入了大模型调用，但由于当前项目以功能验证和小规模原型场景为主，整体调用量可控，因此不会形成过高的运营压力。从维护角度看，系统采用较清晰的模块化结构，便于后续继续扩展推荐算法、社区规则和后台能力。

7.3 总结

本文结合当前已经实现的个性化音乐推荐系统，围绕研究背景、相关技术、系统分析、系统设计、系统实现和系统测试六个层面展开，完成了一套基于 Spring Boot + Vue 的个性化音乐推荐系统论文初稿。论文以六个核心用例为主线，将第三章用例规约、第四章活动图和时序图、第五章功能模块实现以及第六章测试设计进行统一组织，保证了论文结构和项目功能之间的对应关系。

从系统实现效果看，本项目完成了基础音乐推荐系统的常规能力，还加入了基于音频上传的 AI 分析推荐、收藏夹、社区互动、后台 AI 管理等扩展功能，使其较好地体现了内容分析、交互反馈和平台运营三者结合的系统特征。

7.4 展望

尽管当前系统已具备较完整的功能框架，但仍存在进一步扩展空间。第一，可进一步引入更稳定的音频转写与低层音频特征处理工具，增强 AI 分析结果的专业性与稳定性。

第二，可在推荐算法中加入用户行为建模、情感标签、知识图谱或混合推荐策略，进一步提升个性化能力。第三，可完善社区审核规则、推荐解释生成和可视化分析能力，增强系统的产品完整性。第四，可在后续工作中补充性能测试、安全测试与大规模数据集评估，使系统在学术研究和工程落地层面都具备更强说服力。

参考文献

- [1] 张雪,江积海.推荐算法如何赋能内容平台商业模式:“内容为王”还是“社区至上”?——基于网易云音乐的纵向案例研究[J].外国经济与管理,2025,47(06):87-102.
- [2] Kan Y .Personalized recommendation system for network popular music based on audio feature extraction[J].Journal of Computational Methods in Sciences and Engineering,2025,25(6):4962-4974.
- [3] Liu L ,Kong M ,Cao C , et al.Personalized music recommendation algorithm based on machine learning[J].Multimedia Systems,2025,31(2):166-166.
- [4] Deldjoo Y ,Schedl M ,Knees P .Content-driven music recommendation: Evolution, state of the art, and challenges[J].Computer Science Review,2024,51100618-.
- [5] 王元中,王永滨.中西方传统音乐的音乐情感识别特征的对比与分析[J].网络新媒体技术,2025,14(03):54-61.
- [6] 黄川林,鲁艳霞.基于协同过滤和标签的混合音乐推荐算法研究[J].软件工程,2021,24(04):10-14.
- [7] 刘羚瑶.融合音频文本的多模态音乐情感联合学习识别 [J]. 安徽大学学报(自然科学版), 2026, 50 (02): 25-33.
- [8] 刘颖.基于情感分析的音乐推荐算法[D].天津财经大学,2024.
- [9] 毛庆航.基于情感分析的个性化音乐推荐系统的设计与实现[D].曲阜师范大学,2023.
- [10] 潘晓晖,彭炜烨.改进 FP-Growth 算法在音乐推荐中的应用研究[J].信息系统工程, 2021, (08):129-133.
- [11] 刘昊.基于深度学习和协同过滤的音乐推荐系统研究[D].吉林建筑大学,2023.
- [12] 姚勇林.基于深度学习的个性化音乐推荐系统[D].电子科技大学,2023.
- [13] 宋雪峰.基于深度学习的个性化音乐推荐系统设计与实现[D].黑龙江大学,2021.
- [14] 余莉娟.基于深度学习的个性化音乐推荐算法研究[J].微型电脑应用,2020,36(10):140-143.
- [15] 余琳,姜囡,谭浙予,等.基于 SPCCA 的语音和面部表情多模态情感识别[J].中国人民公安大学学报(自然科学版),2025,31(03):51-59.
- [16] 王思阳.基于标签特征的长尾音乐推荐系统设计与实现[D].北京邮电大学,2022.
- [17] 杨明远.基于知识图谱的个性化音乐推荐系统的设计与实现[D].沈阳工业大学,2024.

致 谢

在论文的撰写过程中，要衷心感谢所有曾经给予支持和帮助的人们，感谢导师们，你们的悉心指导、教诲以及在过程中您不辞辛劳地答疑解惑令人受益匪浅，在整个研究过程中给予的耐心指导和宝贵的建议，才能够顺利完成论文的写作，你们的专业知识和丰富经验对系统的开发起到了至关重要的作用。

感谢家人和朋友们，是你们给予了鼓励和支持；在系统开发的过程中给予的无条件支持和理解使得论文顺利的进行了下去，他们在遇到困难和挫折时给予的鼓励和支持，使得能够坚定地走下去，在困难时刻坚持不懈、勇往直前。

感谢所有曾经给予过建设性意见和批评性评价的同学和老师，你们的意见和建议让系统开发更加完善；感谢所有在完成系统开发中给予帮助和支持的人们，没有你们的支持和鼓励，将无法完成这篇论文。在此要向所有给予过支持和帮助的人们表示最诚挚的感谢和敬意，无论是直接还是间接的支持和帮助，都受益匪浅；感谢你们的帮助和支持，愿在今后的人生中能够继续努力，取得更好的成绩。

